

Embedded System Design

Introduction and UNIT 1

Anuj Kumar Sharma
Assistant Professor
Dept. of ECE
FET, GKV
Haridwar

Index

- Discussion of Syllabus.
- Books and material to be study.
- UNIT First Start
 1. Introduction of Embedded System Design and microcontrollers
 2. Embedded Operating system.
 3. Design parameters of an embedded system and its significance
 4. Design life cycle.
 5. Hardware and Software partitioning and co-design
 6. Hardware Fundamentals for the embedded developers.
 7. Open collector outputs Tristate outputs I/O sinking and Sourcing.
 8. PLD's, Watchdog Timers.

Syllabus

[embedded system design syllabus.docx](#)

[Embedded system Design Lab syllabus.docx](#)

Prerequisite

You should have some fundamental knowledge of
Microprocessors.

Books and material to be study

- **Text Book**

- Embedded System Design-Frank Vahid/Tony Givargis, John Willey, 2005.

- **References Books**

- 1. The 8051 Microcontroller and embedded systems-Muhammad Ali Mazidi and Janice Gillispie.
- 2. Microcontrollers (Architecture, Implementation & Programming) Kenneth Hintz, Daniel Tabak, Tata McGraw-Hill, 2005.
- 3. 8051 Microcontrollers & Embedded Systems 2nd Edition-Sampath Kr, Katson Books, 2006.
- 4. Microcontroller (Theory and Applications) Ajay V Deshmukh, Tata McGraw-Hill, 2005.
- 5. An Embedded Software Primer-David E.Simon, Pearson Education, 1999.

Introduction of Embedded system Design

**Embedded
System
Design**

1. Embedded

- An embedded system is a computer system—a combination of a computer processor, computer memory, and input/output peripheral devices—that has a dedicated function within a larger mechanical or electrical system.
- It is embedded as part of a complete device often including electrical or electronic hardware and mechanical parts. Because an embedded system typically controls physical operations of the machine that it is embedded within.

Embedded Components

- 1. Embedded Hardware
- 2. Embedded main Application Software
- 3 Embedded real time operating System

2. System

- system.jpeg

3. Design

- Software partitioning
- Hardware partitioning

4. Embedded system

- **What is an Embedded system?**
- • An embedded system is one that has computer hardware with software embedded in it as one of its components.
- Or We can define an embedded system as “**A microprocessor based system that does not look like a computer**”.
- Or we can say that it is “**A combination of computer hardware and software, and perhaps additional mechanical or other parts, designed to perform a dedicated function. In some cases, embedded systems are part of a larger system or product, as is the case of an antilock braking system in a car**”.

- 
- Or An **embedded system** is a **special-purpose computer system** designed to perform certain dedicated functions. It is usually embedded as part of a complete device including hardware and mechanical parts.

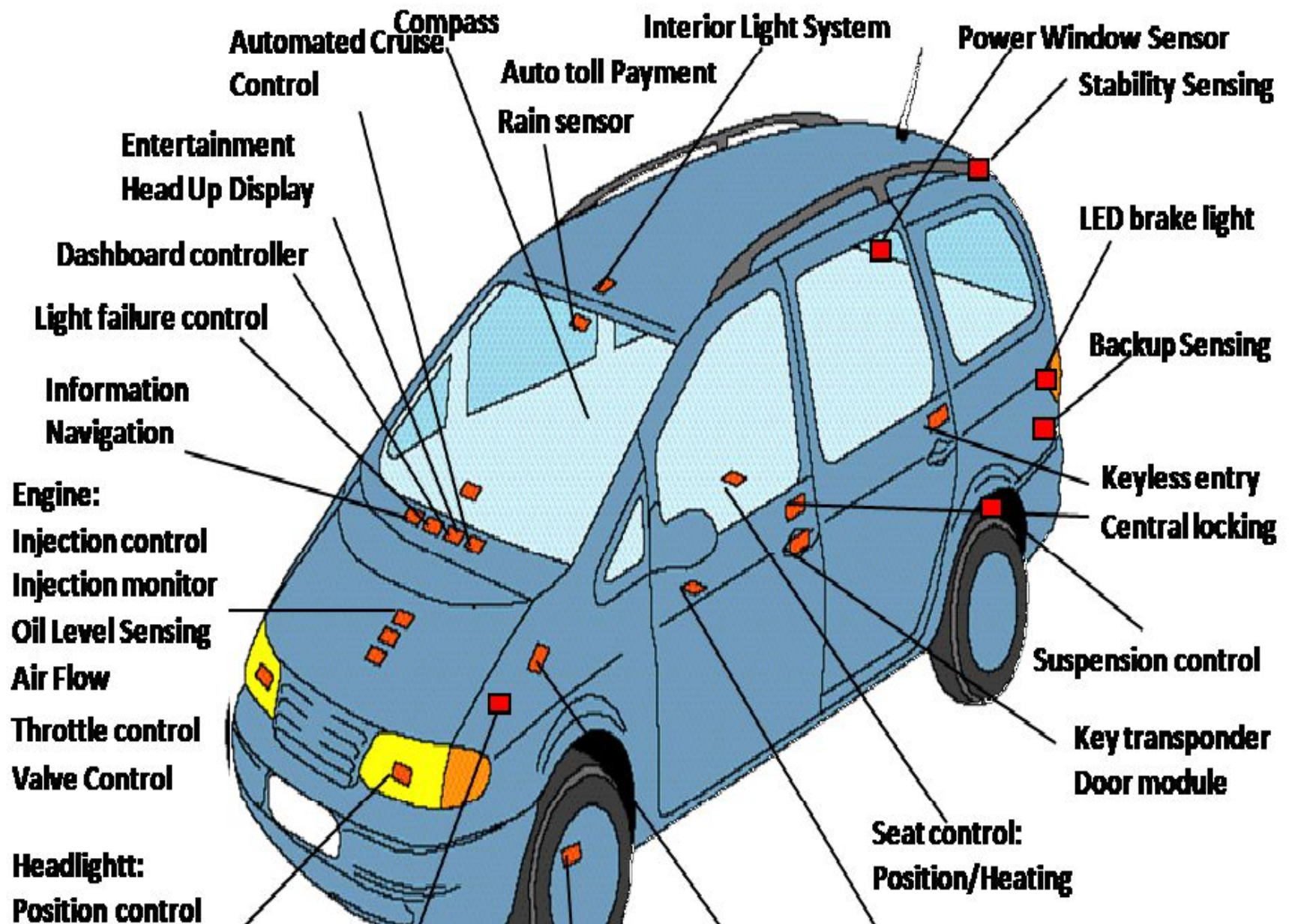
5. Significance of an Embedded System

- Due to their compact size, low cost and simple design aspects made embedded systems very popular and encroached into human lives and have become indispensable. They are found everywhere from kitchen ware to space craft.

- Embedded systems span all aspects of modern life and there are many examples of their use.
- a) Biomedical Instrumentation – ECG Recorder, Blood cell recorder, patient monitor system.
- b) Communication systems – pagers, cellular phones, cable TV terminals, fax and transreceivers, video games and so on.
- c) Peripheral controllers of a computer – Keyboard controller, DRAM controller, DMA controller, Printer controller, LAN controller, disk drive controller.
- d) Industrial Instrumentation – Process controller, DC motor controller, robotic systems, CNC machine controller, close loop engine controller, industrial moisture recorder and controller.
- e) Scientific – digital storage system, CRT display controller, spectrum analyzer.

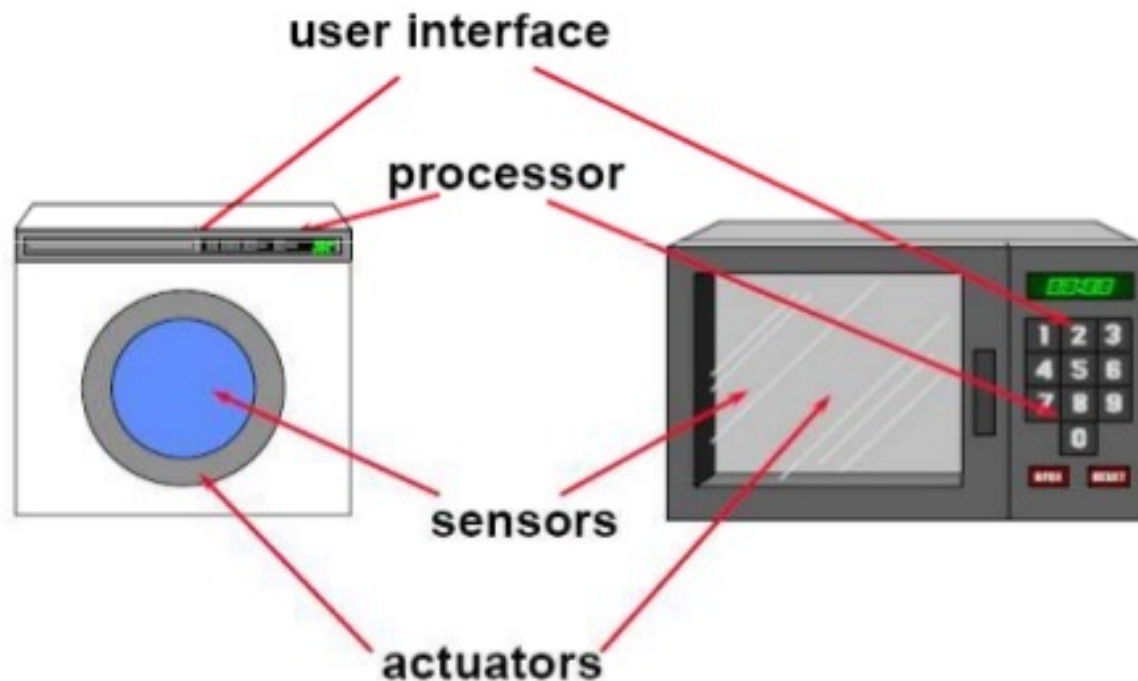
6. Example of some basic embedded system



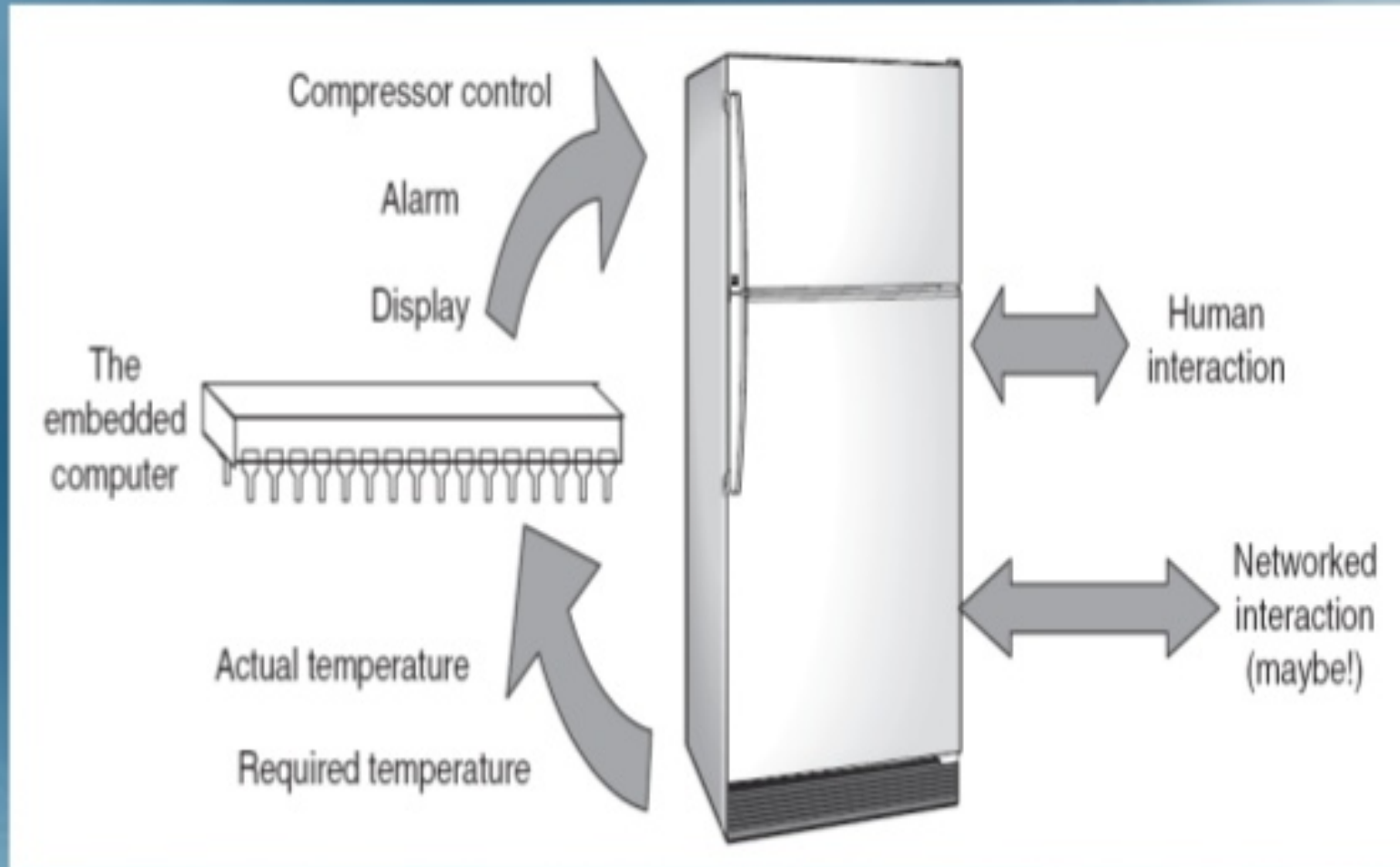


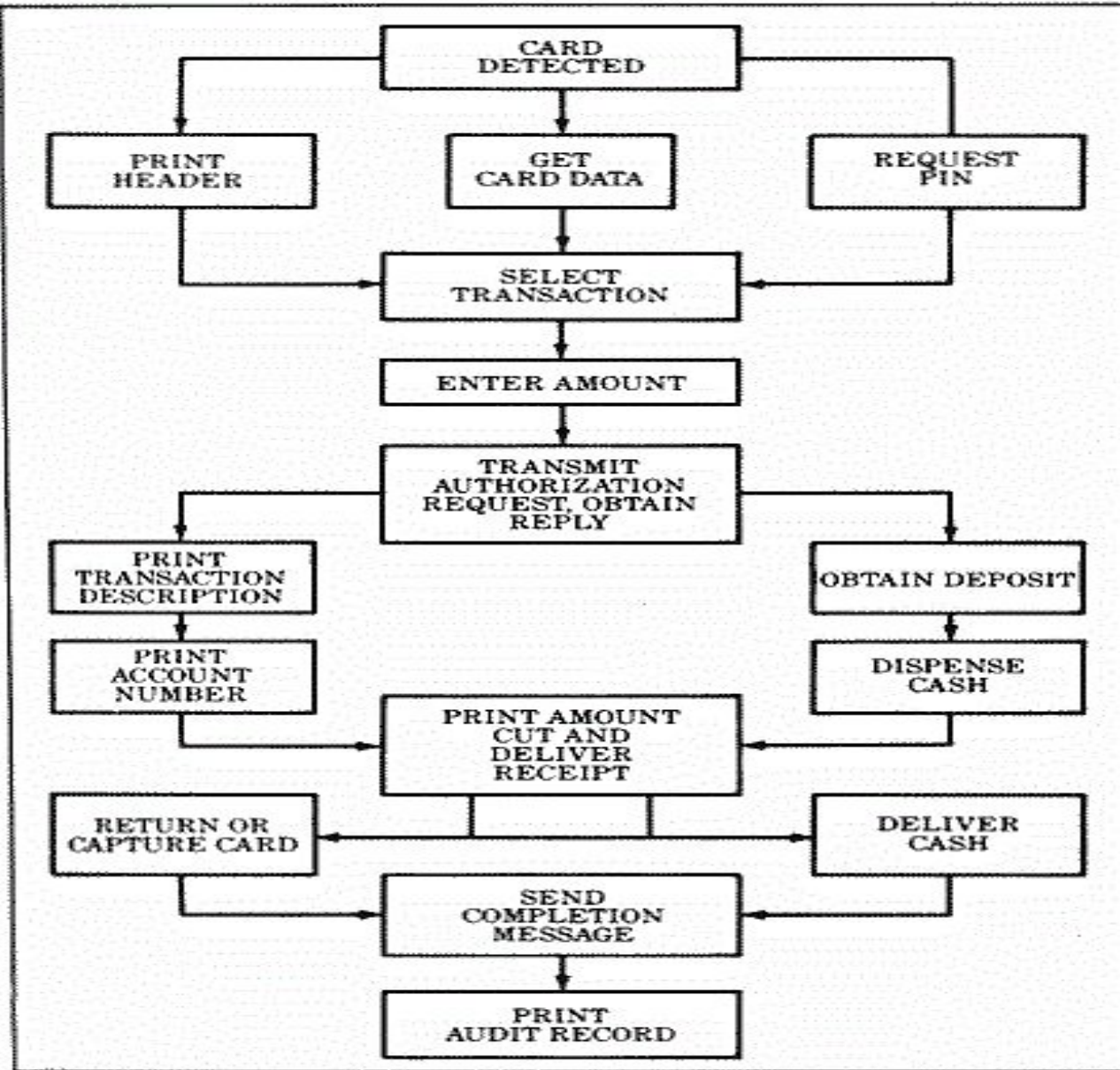
Examples of Embedded Systems

Consumer electronics, for example MP3 Audio, digital camera, home electronics,

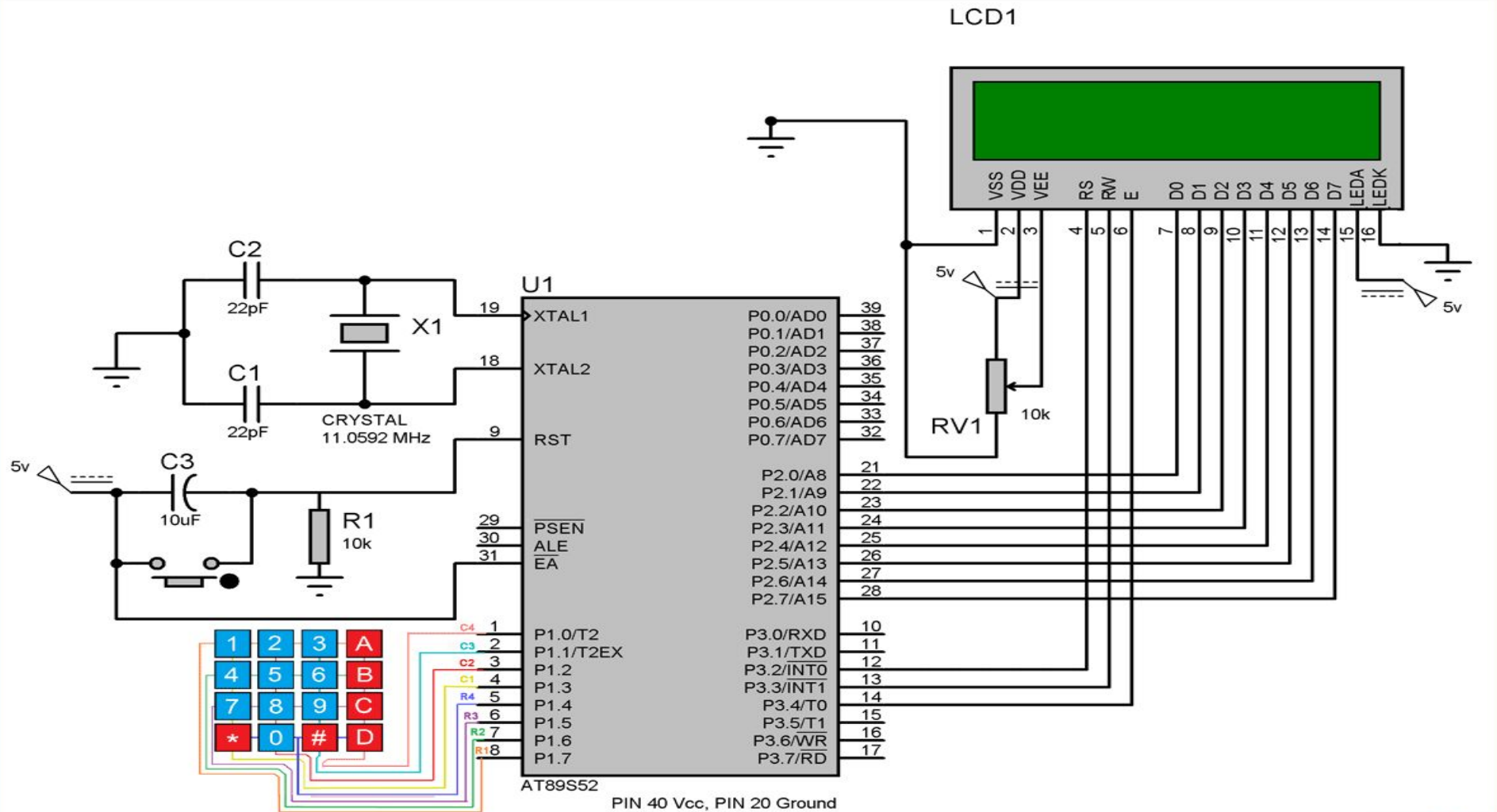


Example of Embedded System





ATM MACHINE



Keypad and LCD interfacing

7. Computer system Vs embedded system

< General Purpose PC >



< Embedded System >

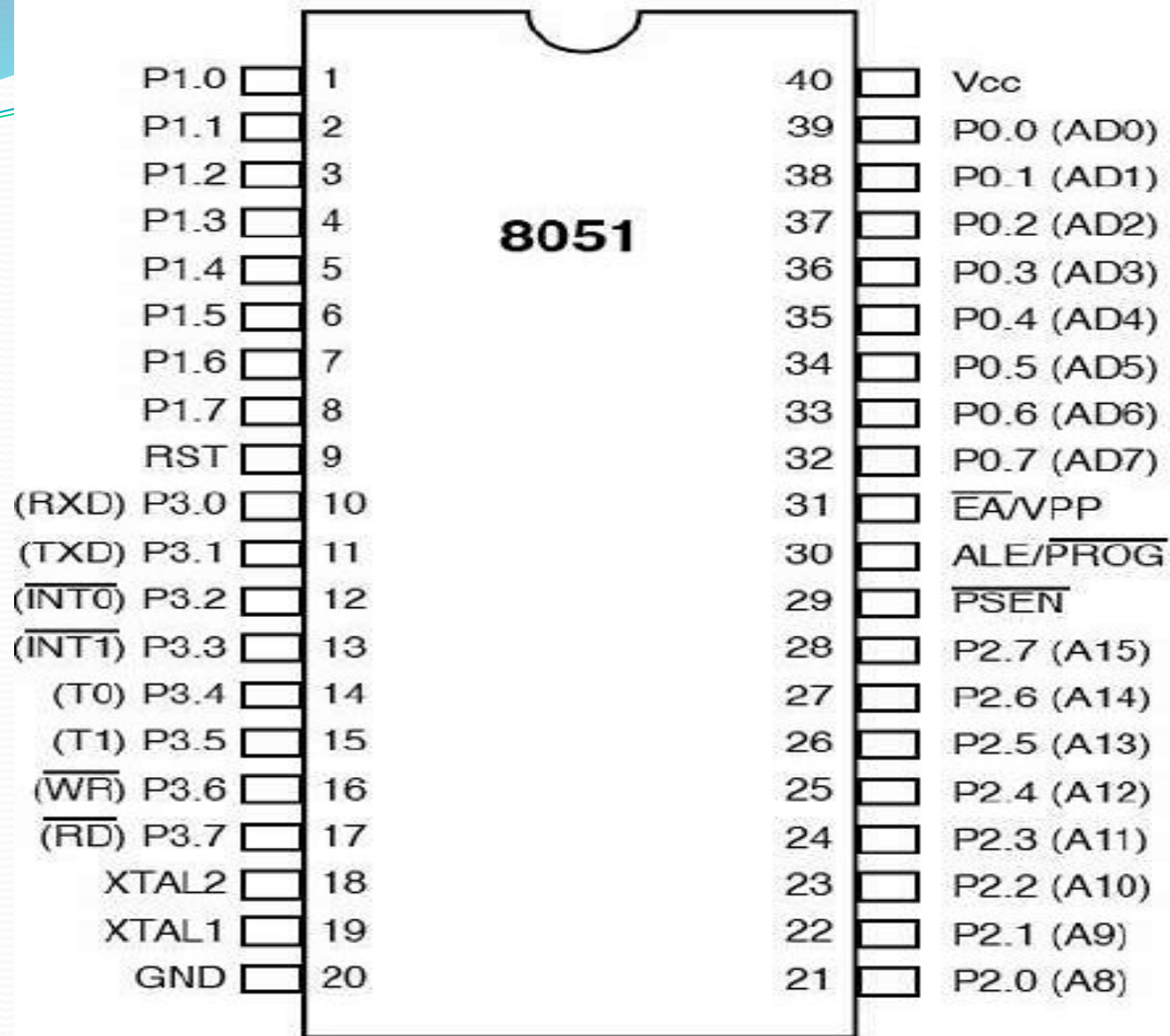


8. Were the embedded systems existing earlier ?

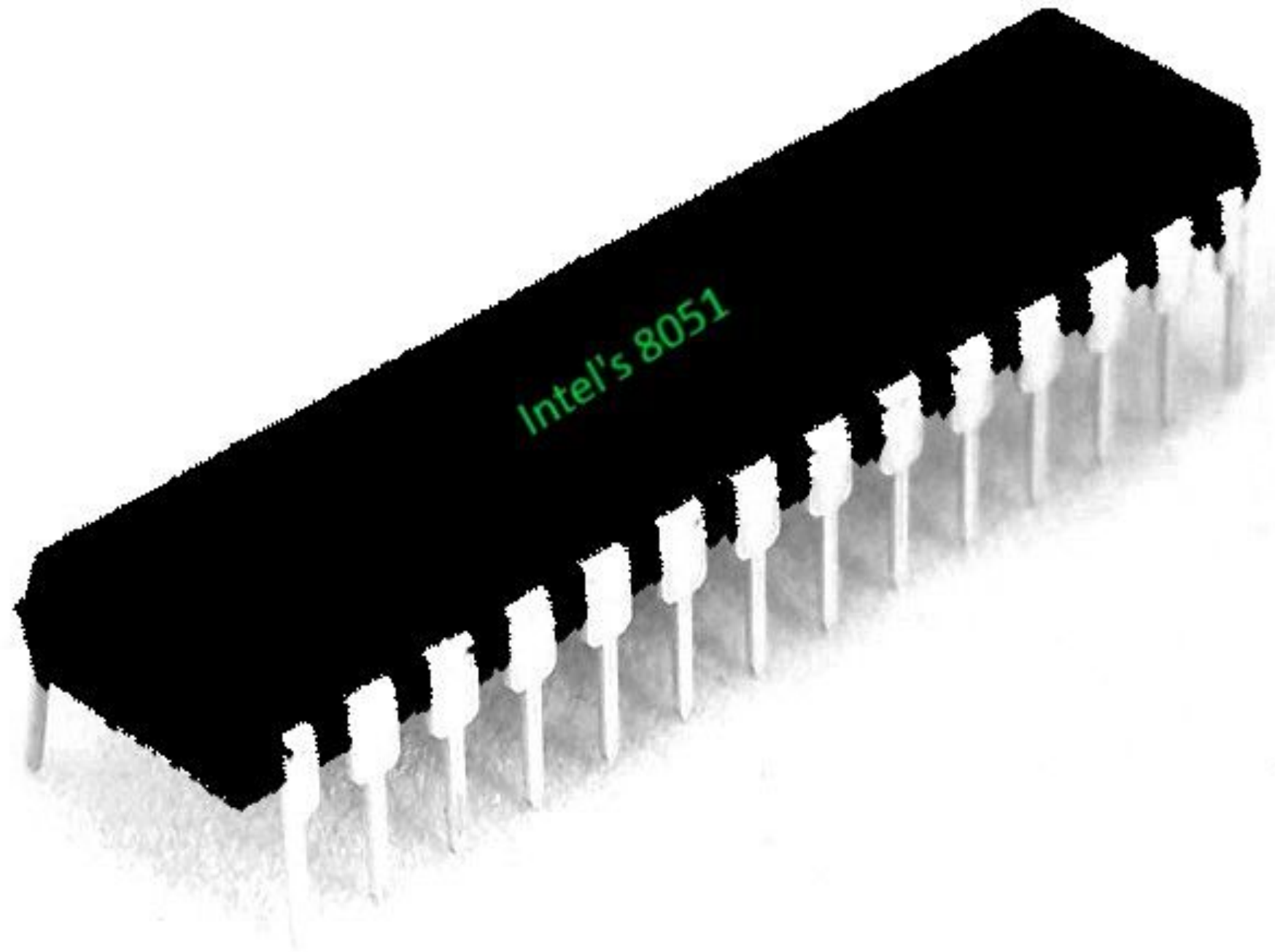
- Yes, We have been enjoying the grace of embedded system quite a long time. But they were not so popular because in those days most of the embedded systems were designed around a microprocessor unlike today's systems which were built around a microcontroller.
- As we know a microprocessor by itself do not possess any memory, ports etc. So everything must be connected externally by using peripherals like 8255, 8257, 8259 etc. So the embedded system designed using microprocessor was not only complicated in design but also large in size. At the same time the speed of microprocessor is also a limitation for high end applications.

9. Why a microcontroller ?

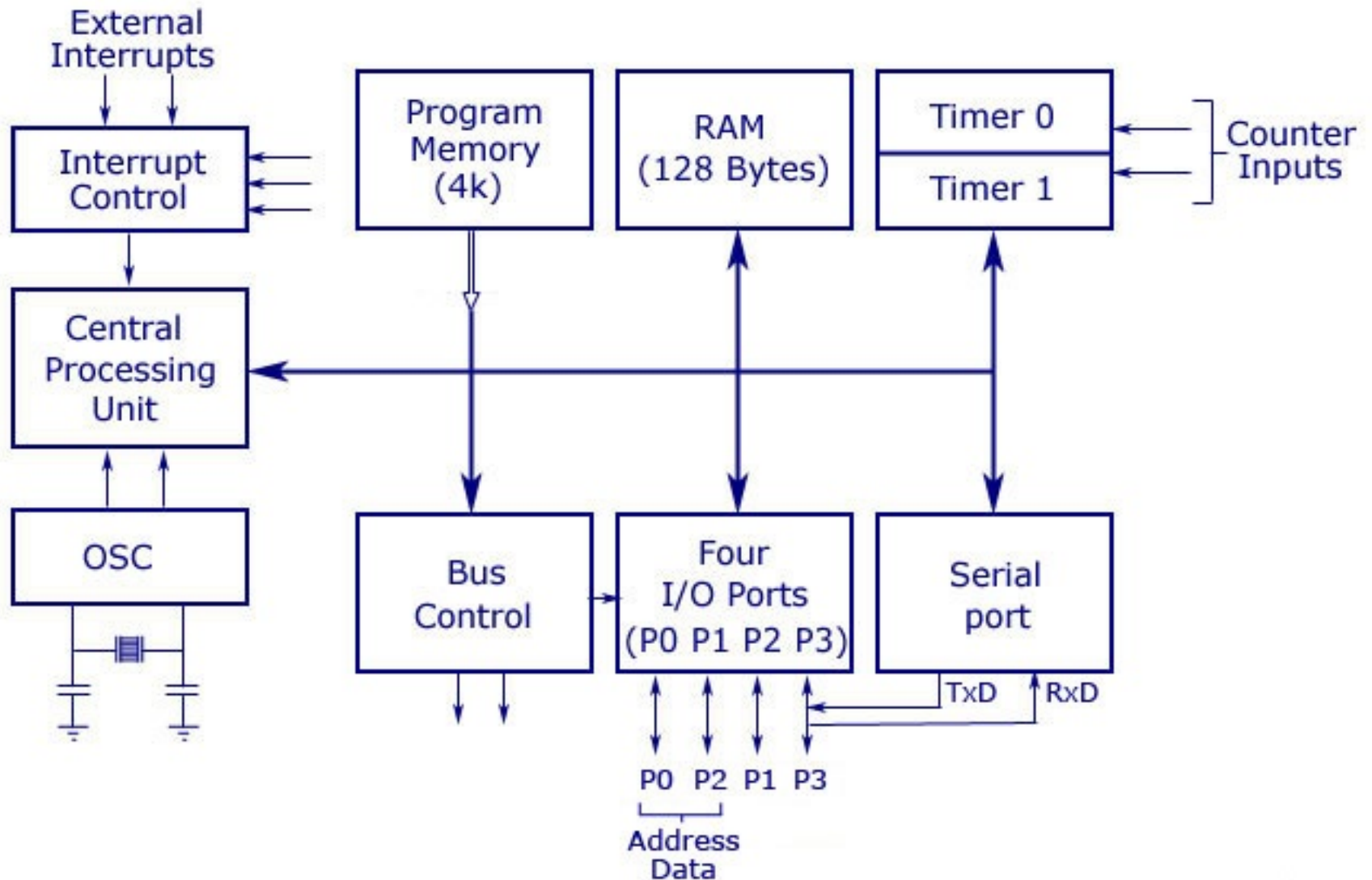
- A microcontroller is a single silicon chip with memory and all Input/Output peripherals on it. Hence a microcontroller is also popularly known as a single chip computer. Normally, a single microcomputer has the following features :
- 1. Arithmetic and logic unit
- 2. Memory for storing program
- 3. EEPROM for nonvolatile data storage
- 4. RAM for storing variables and special function registers
- 5. Input/output ports
- 6. Timers and counters
- 7. Analog to digital converter
- 8. Circuits for reset, power up, serial programming, debugging
- 9. Instruction decoder and a timing and control unit
- 10. Serial communication port



Pin Diagram of microcontroller 8051



Simplified Internal Architecture of XX51



10. Classification of Embedded Systems

- Based on functionality and performance requirements, embedded systems are classified as :
 - 1. Stand-alone Embedded Systems
 - 2. Real-time Embedded Systems
 - 3. Networked Information Appliances
 - 4. Mobile Devices

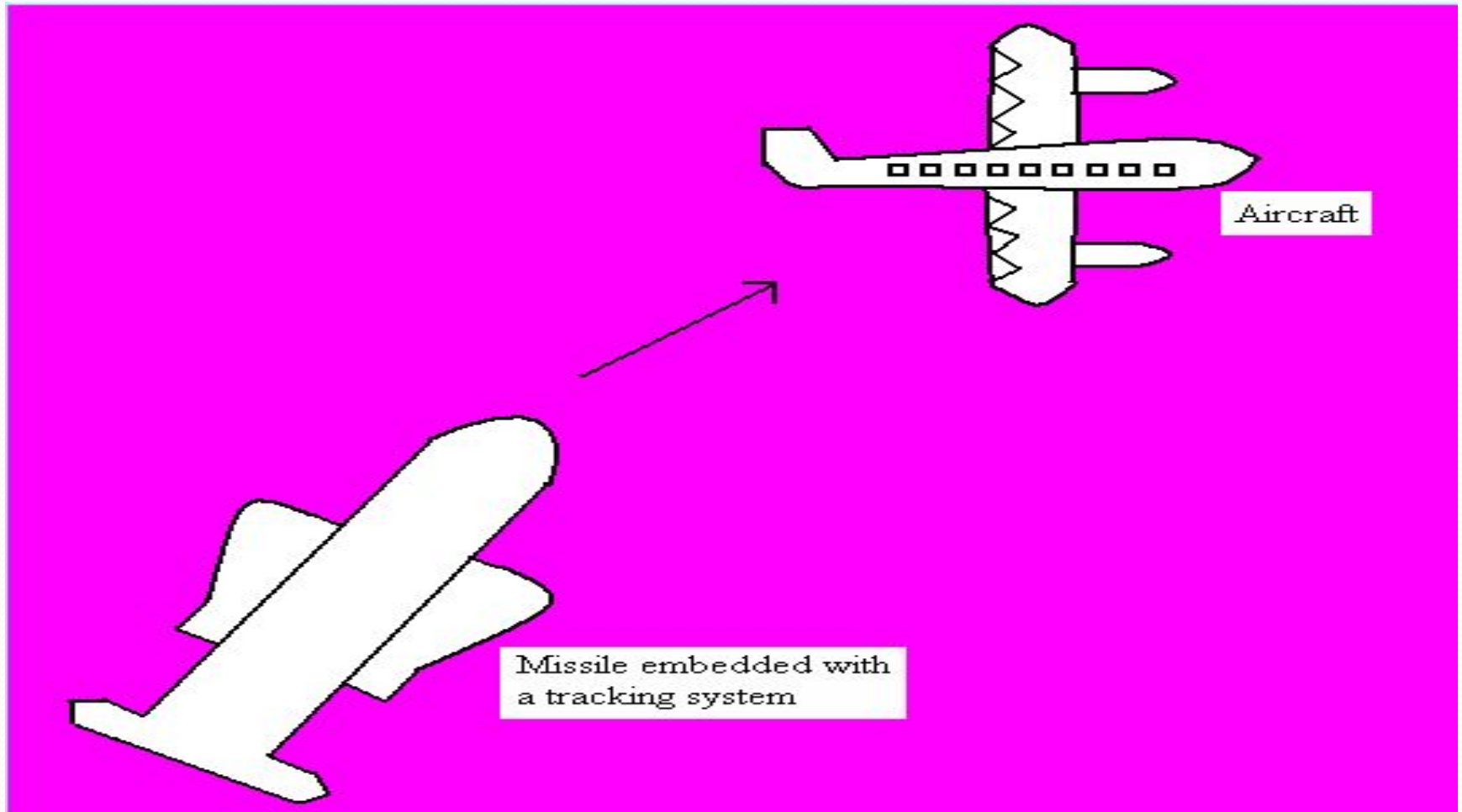
Stand-alone Embedded Systems

- As the name implies, stand-alone systems work in stand-alone mode. They take inputs, process them and produce the desired output. The input can be electrical signals from transducers or commands from a human being such as the pressing of a button. The output can be electrical signals to drive another system, an LED display or LCD display for displaying of information to the users. Embedded systems used in process control, automobiles, consumer electronic items etc. fall into this category.

Real-time Systems

- Embedded systems in which some specific work has to be done in a specific time period are called real-time systems. For example, consider a system that has to open a valve within 30 milliseconds when the humidity crosses a particular threshold. If the valve is not opened within 30 milliseconds, a catastrophe may occur. Such systems with strict deadlines are called *hard real-time systems*.
- In some embedded systems, deadlines are imposed, but not adhering to them once in a while may not lead to a catastrophe. For example, consider a DVD player. Suppose, you give a command to the DVD player from a remote control, and there is a delay of a few milliseconds in executing that command. But, this delay won't lead to a serious implication. Such systems are called *soft real-time systems*.

Hard Real-Time Embedded System

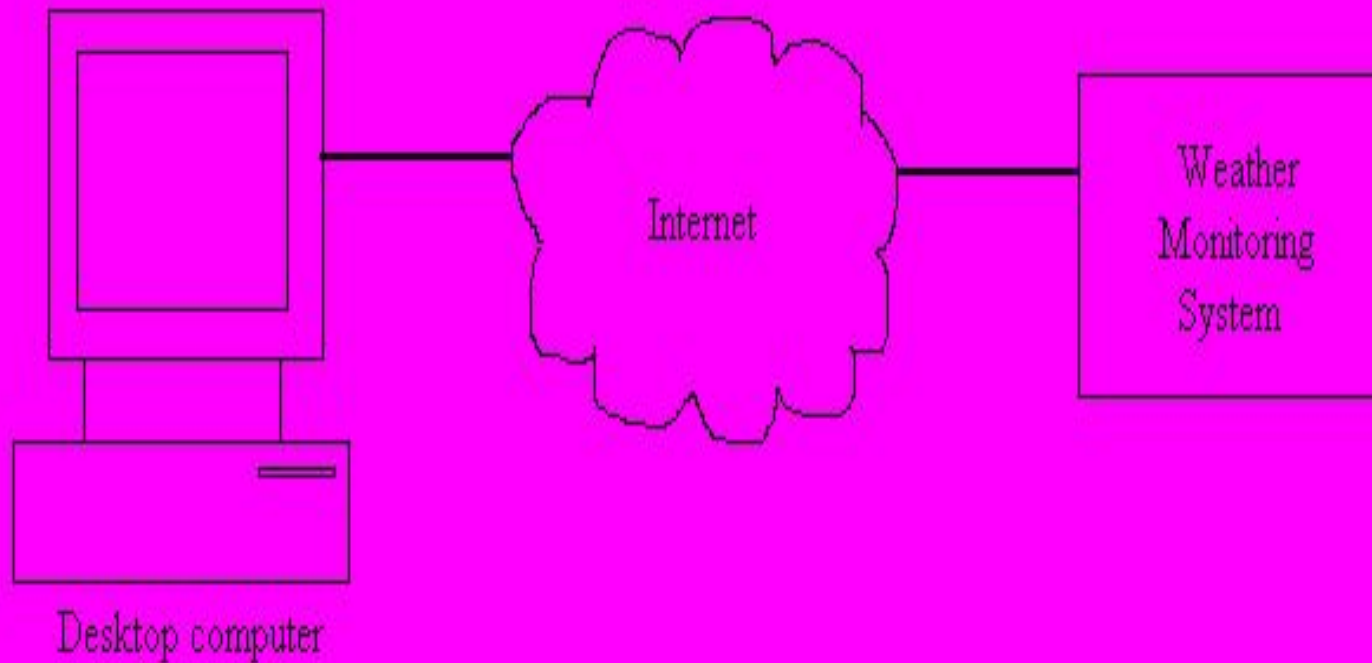


Networked Information Appliances

- Embedded systems that are provided with network interfaces and accessed by networks such as Local Area Network or the Internet are called networked information appliances. Such embedded systems are connected to a network, typically a network running TCP/IP (Transmission Control Protocol/Internet Protocol) protocol suite, such as the Internet or a company's Intranet.
- These systems have emerged in recent years. These systems run the protocol TCP/IP stack and get connected through PPP or Ethernet to an network and communicate with other nodes in the network.

Example of Networked Information Appliances

- A networked process control system consists of a number of embedded systems connected as a local area network. Each embedded system can send real-time data to a central location from where the entire process control system can be monitored. The monitoring can be done using a web browser such as the Internet Explorer.
- A web camera can be connected to the Internet. The web camera can send pictures in real-time to any computer connected to the Internet. In such a case, the web camera has to run the HTTP server software in addition to the TCP/IP protocol stack.
- The door lock of your home can be a small embedded system with TCP/IP and HTTP server software running on it. When your children stand in front of the door lock after they return from school, the web camera in the door-lock will send an alert to your desktop over the Internet and then you can open the door-lock through a click of the mouse.



- Here we saw a weather monitoring system connected to the Internet. TCP/IP protocol suite and HTTP web server software will be running on this system. Any computer connected to the Internet can access this system to obtain real-time weather information.

Mobile Devices

- Mobile devices such as mobile phones, Personal Digital Assistants (PDAs), smart phones etc. are a special category of embedded systems. Though the PDAs do many general purpose tasks, they need to be designed just like the ‘conventional’ embedded systems

10. Languages for Programming Embedded Systems

- Assembly language was the pioneer for programming embedded systems till recently. Nowadays there are many more languages to program these systems. Some of the languages are C, C++, Ada, Forth, and Java together with its new enhancement J2ME.
- The presence of tools to model the software in UML, SDL is sufficient to indicate the maturity of embedded software programming.
- The majority of software for embedded systems is still done in C language.
- Recent survey indicates that approximately 45% of the embedded software is still being done in C language. C++ is also increasing its presence in embedded systems. As C++ is based on C language, thus providing programmer the object oriented methodologies to reap the benefits of such an approach.

- C is very close to assembly programming and it allows very easy access to underlying hardware. A huge number of high quality compilers and debugging tools are available for the C language.
- Though C++ is theoretically more efficient than C, but some of its compilers have bugs due to the huge size of the language. These compilers may cause a buggy execution.
- C language can definitely claim to have more mature compilers C++.
- Now in order to avail the extra benefits of C++ and plus to avoid buggy execution, experts are doing efforts to identify a subset of C++ that can be used in embedded systems and this subset is called Embedded C++

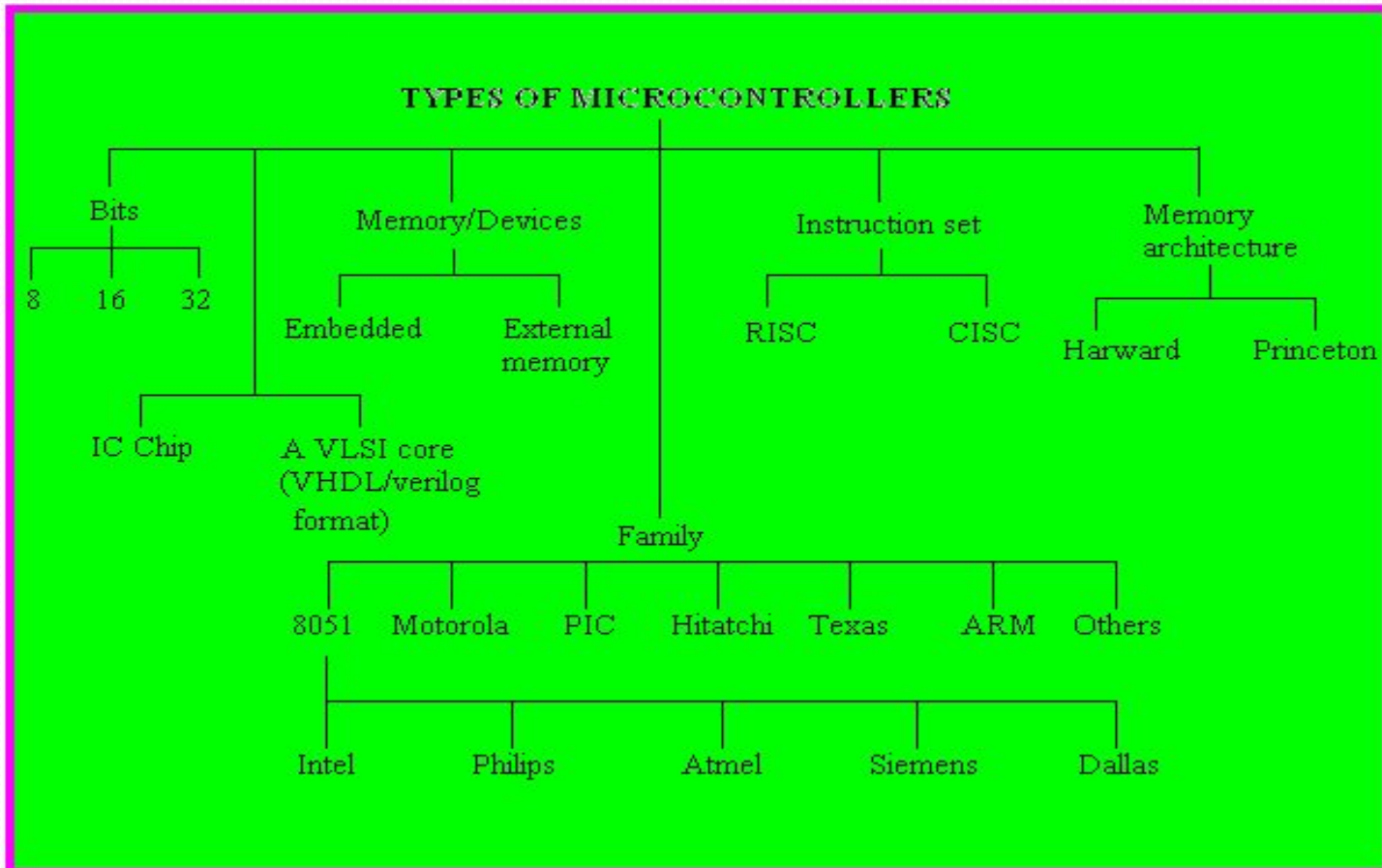
11. Communication Interfaces

- For embedded systems to interact with the external world, a number of
- communication interfaces are available. They are
- 1. Serial Communication Interfaces (SCI): RS-232, RS-422, RS-485 etc
- 2. Synchronous Serial Communication Interface: I2C, JTAG, SPI, SSC and ESSI
- 3. Universal Serial Bus (USB).
- 4. Networks: Ethernet, Controller Area Network, LonWorks, etc
- 5. Timers: PLL(s), Capture/Compare and Time Processing Units
- 6. Discrete IO: General Purpose Input/Output (GPIO)
- 7. Analog to Digital/Digital to Analog (ADC/DAC)

12. Which is the best suited microcontroller for design of embedded system?

- There is always a trade off between efficiency and power dissipation. To know this, let us review the various types of microcontrollers and their specifications and the vendors.

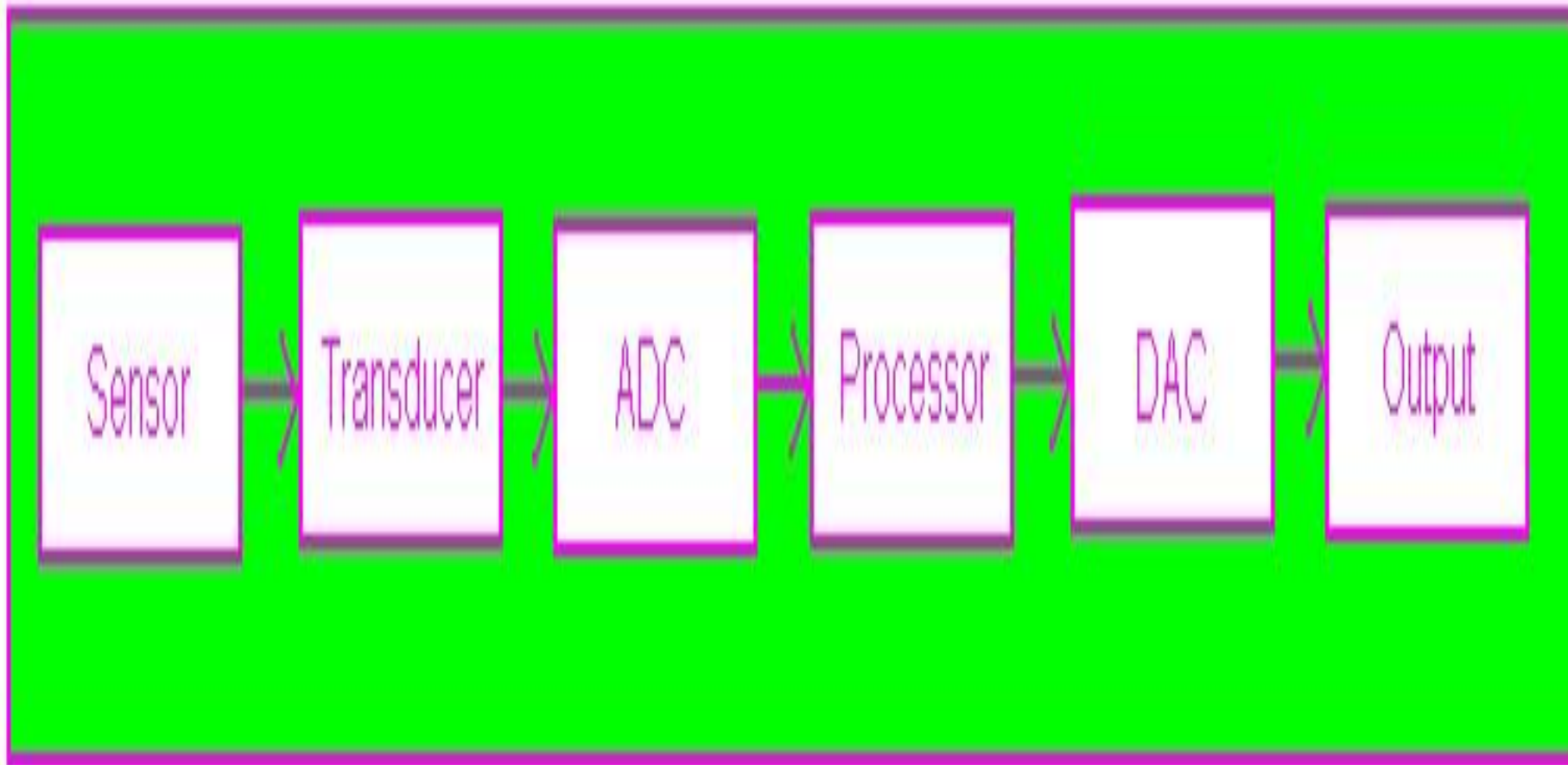
13. Types of Microcontroller



- From the previous slide we can find that the ARM processor is a strong option for better performance. But when we consider the power consumption, in the case of ARM it is around 400mW and the ATmega1031, AVR microcontroller consumes low power around 16.5mW, but provides low performance.
- But the Texas instruments MSP430 with wide range of operation modes consumes only 1.2mW with reasonably good performance. So it is always left to the designer to choose a suitable device according to the requirement.

14. Design of an embedded system – a Case study

- To understand the design of a simple embedded system let us first consider the idea of a data acquisition system. The data acquisition system is shown in the next slide.

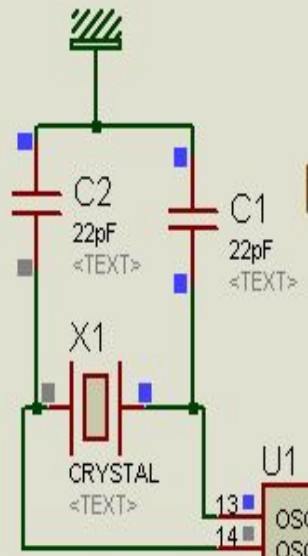
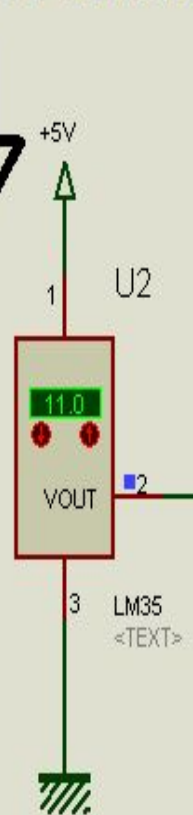
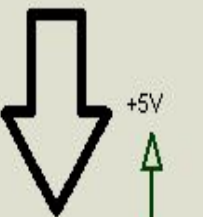


● **Data acquisition system**

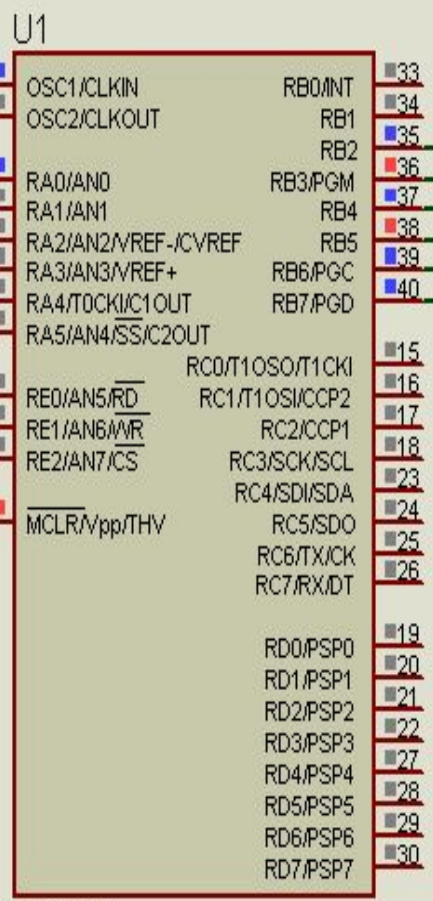
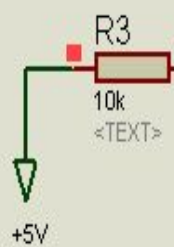
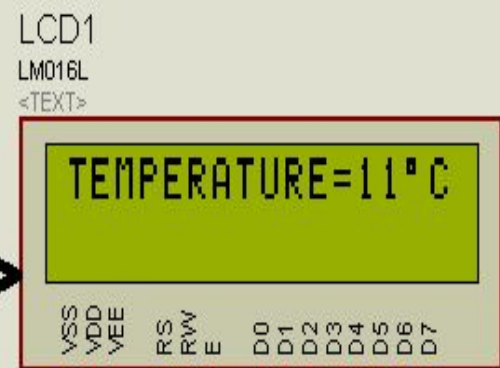
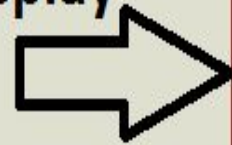
- For example let me consider a simple case of temperature measurement embedded system. First we must select a temperature sensor like thermistor or AD590 or LM35 or LM335 or LM75 etc. After this the analog data is converted into digital data and at the same time proper signal conditioning is done.
- This digital input is fed to the microcontroller through its ports. By developing a suitable program (Embedded C or Assembly) the data is processed and controlled. For this purpose keil or Ride or IAR ARM Embedded workbench C compilers can be used.

- 
- Once the program is debugged, and found error free it can be dumped into the microcontroller flash memory using ISP (Philips - Flash magic or any ISP). Now, your microcontroller chip acts as an embedded chip.

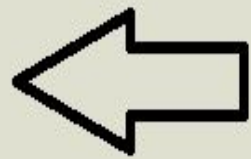
LM35 sensor



LCD with temp display



Microcontroller



PIC16F877A

Hardware and Software requirement to develop any Embedded system

● Hardware Requirement

1. Input devices : Keypad , Sensors
2. Process Devices : Microcontroller, Microprocessor
3. Output Devices : LED's , LCD's , Seven segment Display , DC Motors , ADC, DAC etc.
4. other devices , Crystal oscillator, ROM burner(any flash device), resistors, capacitors, Connecting wires.

● Software Requirement

1. Any Flash device Interfacing Software
2. Any compiler to compile Embedded program like Keil compiler
3. Proteus for design embedde system on software platform

Introduction to Keil Compiler and Proteus

- The Proteus Design Suite is a proprietary software tool suite used primarily for electronic design automation. The software is used mainly by electronic design engineers and technicians to create schematics and electronic prints for manufacturing printed circuit boards.
- The **Keil C51 C Compiler** for the 8051 microcontroller is the most popular 8051 C **compiler** in the world.
- The C51 **Compiler** translates C source files into relocatable object modules which contain full symbolic information for debugging with the μ Vision Debugger or an in-circuit emulator.

First LEB Experiment

- Program to interface LED display unit and implement it on software and hardware

Components Required (Software and Hardware)

- 1. LEDS
- 2 Crystal oscillator
- 3 Microcontroller 89C51
- 4 Capacitors
- 5 Connecting wires
- 6 ROM burnner
- 7 Kiel Compiler
- 8 Proteus

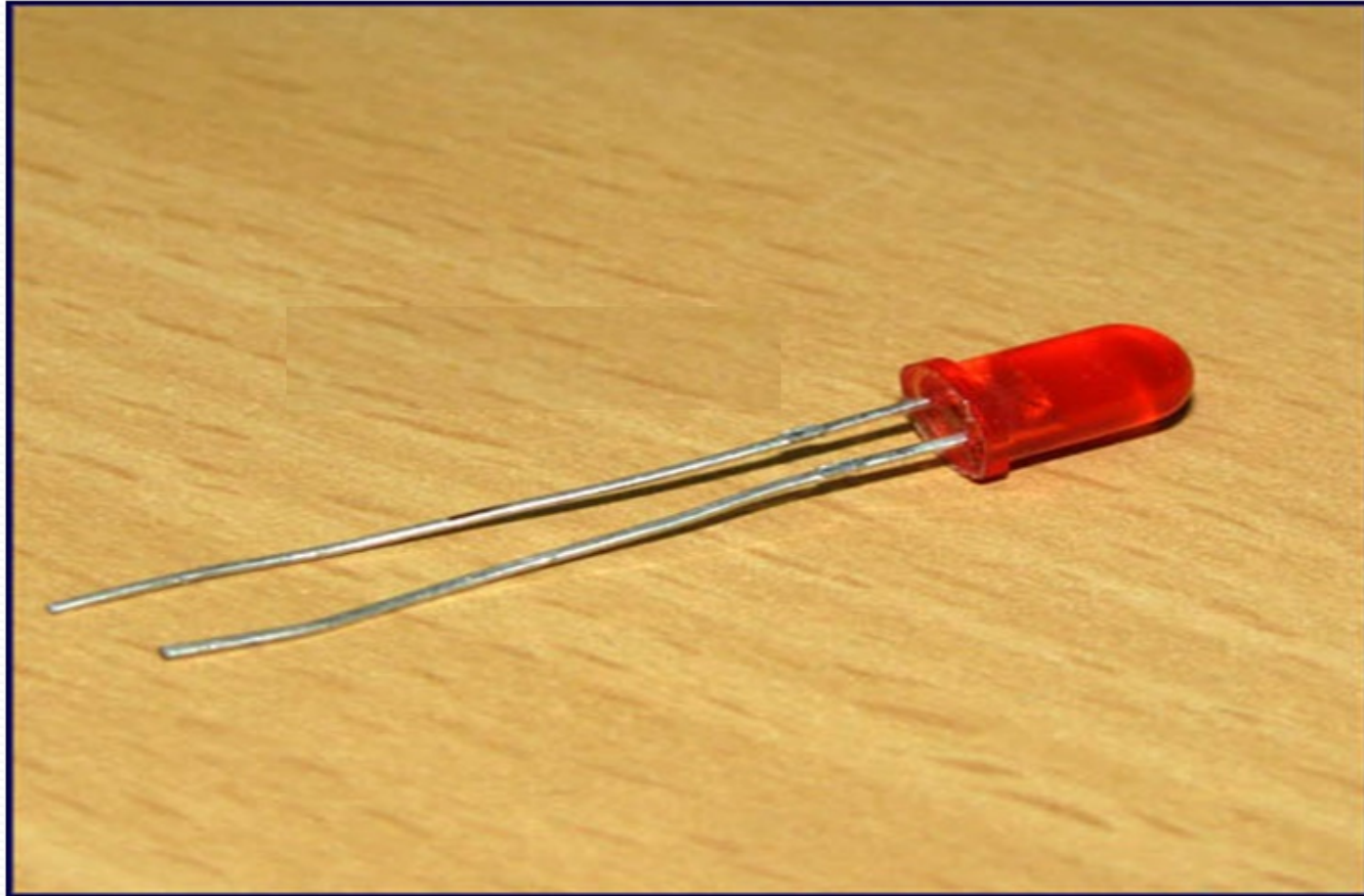
Lab Manual

- LEDs are by far the most widely used means of taking output. They find huge application as indicators during experimentations to check the validity of results at different stages. They are very cheap and easily available in a variety of shape, size and colors.
- The principle of operation of LEDs is simple. The commonly available LEDs have a drop voltage of 1.7 V and need 10 mA to glow at full intensity. The following circuit describes “how to glow an led”.
- The value of resistance R can be calculated using the equation, $R = (V - 1.7) / 10 \text{ mA}$. Since most of the controllers work on 5V, so substituting $V = 5V$, the value of resistance comes out to be 330 ohm. The resistance 220 ohm, 470 ohm is commonly used substitute in case 330 ohm is not available.
- AT89C51 is a 40 pin microcontroller which belongs to 8051 series of microcontroller. It has four ports each of 8 bits P0, P1, P2 and P3. The AT89C51 has 4K bytes of programmable flash. The port P0 covers the pin 32 to pin 39, the port P1 covers the pin 1 to pin 8, the port P2 covers the pin 21 to pin 28 and the port P3 covers the pin 10 to pin 17. Pin 9 is the reset pin. The reset is active high. Whenever the controller is given supply, the reset pin must be given a high signal to reset the controller and bring the program counter to the starting address 0x0000. The controller can be reset by manually connecting a switch or by connecting a combination of resistor and capacitor as shown in the circuit diagram. A 12 MHz crystal is connected between pin 18 pin 19. Pin 40 is Vcc and pin 20 is ground. Pin 31, is connected to Vcc as we are using the internal memory of the controller.

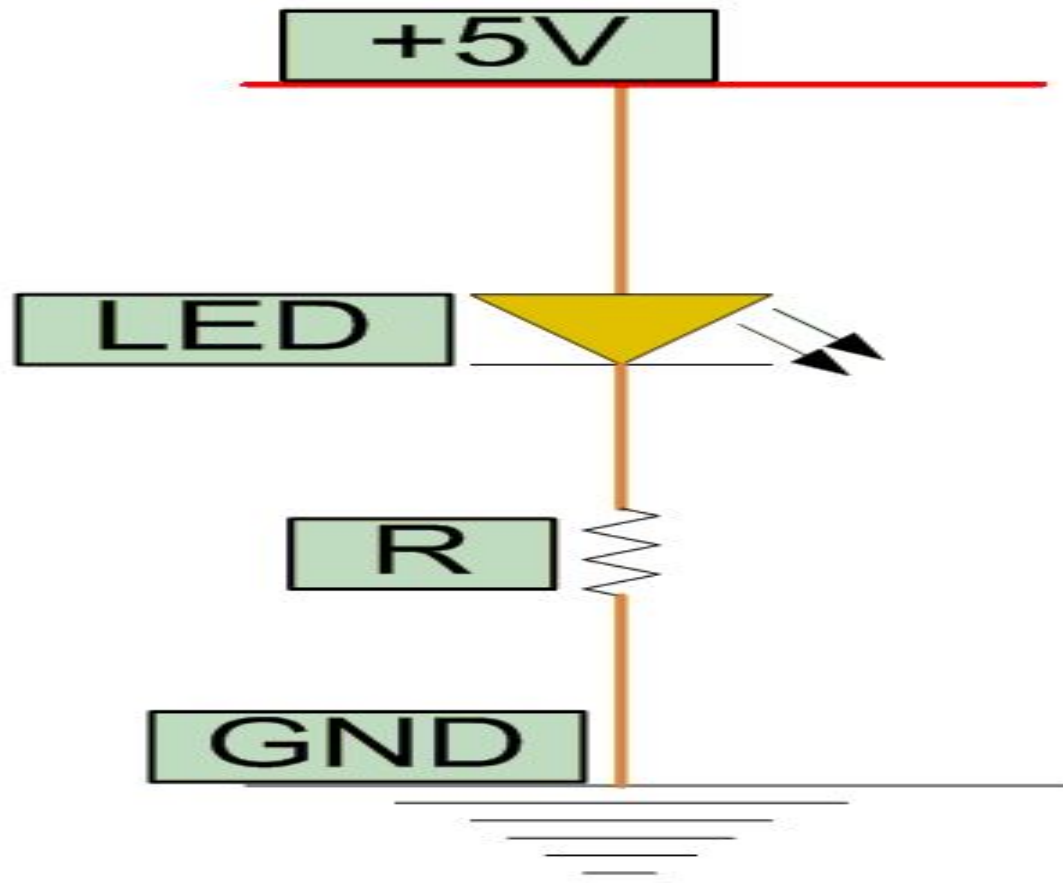
Cont...

- LEDs are connected to the port P0. LEDs need approximately 10mA current to flow through them in order to glow at maximum intensity. However the output of the controller is not sufficient enough to drive the LEDs, so if the positive leg of the LED is connected to the pin and the negative to ground as shown in the figure, the LED will not glow at full illumination.
- To overcome this problem LEDs are connected in the reverse order and they run on negative logic i.e., whenever 1 is given on any pin of the port, the LED will switch off and when logic 0 is provided the LED will glow at full intensity.
- As soon as we provide supply to the controller, the LEDs start blinking i.e., they become on for a certain time duration and then become off for the same time duration. This delay is provided by calling the delay function. The values inside the delay function have been set to provide a delay in multiples of millisecond (delay (100) will provide a delay of 100 millisecond).

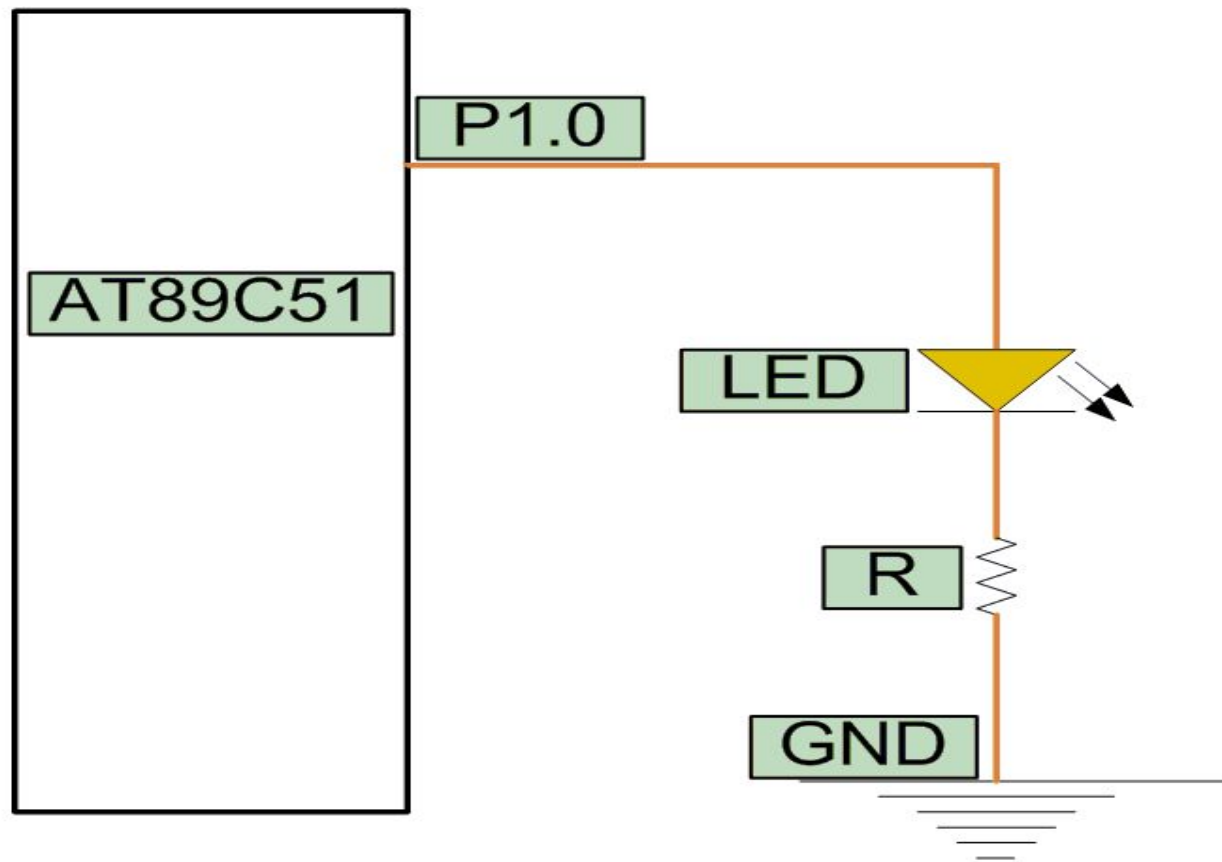
LED



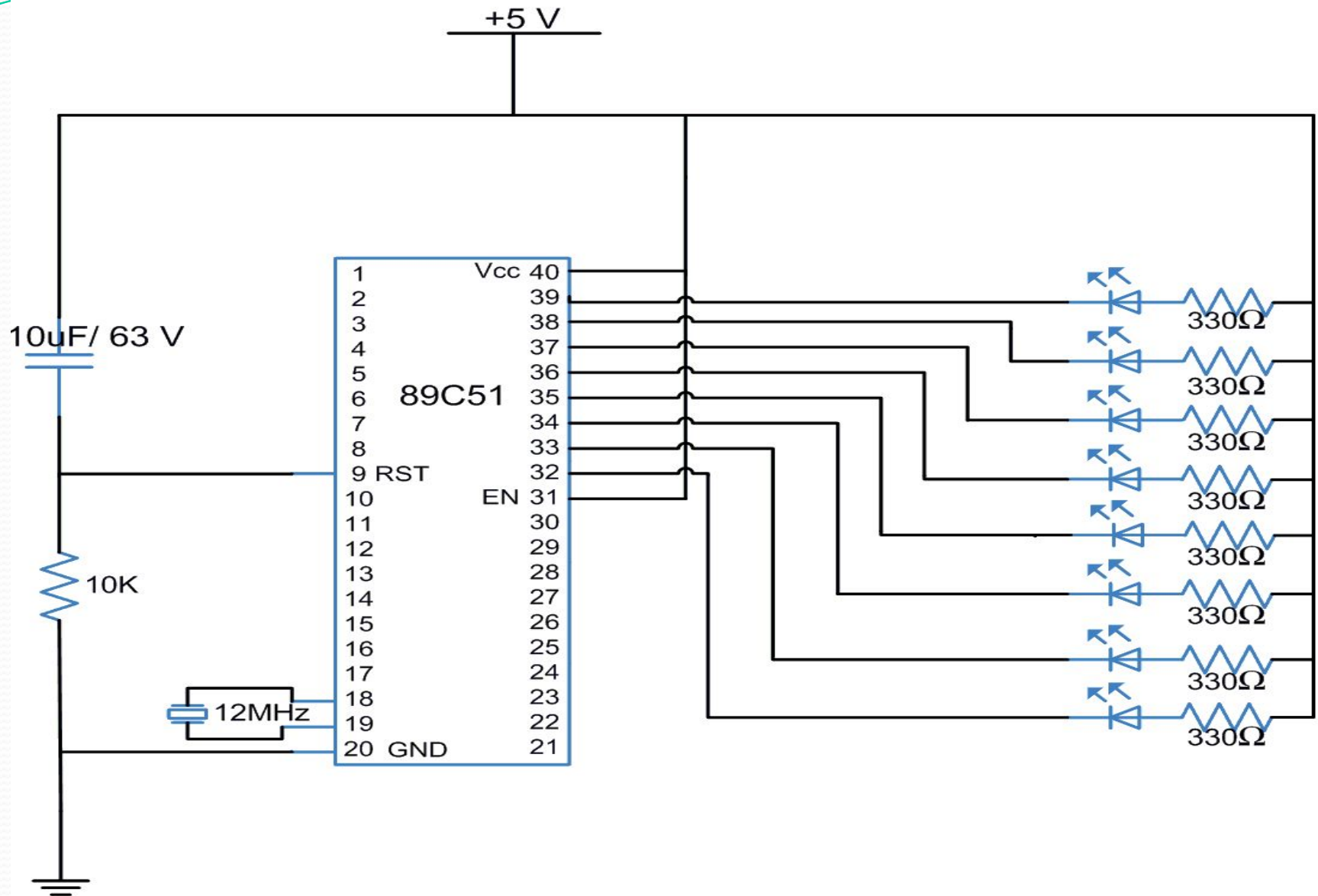
How a LED work



LED Interfacing with 89C51



Circuit Diagram



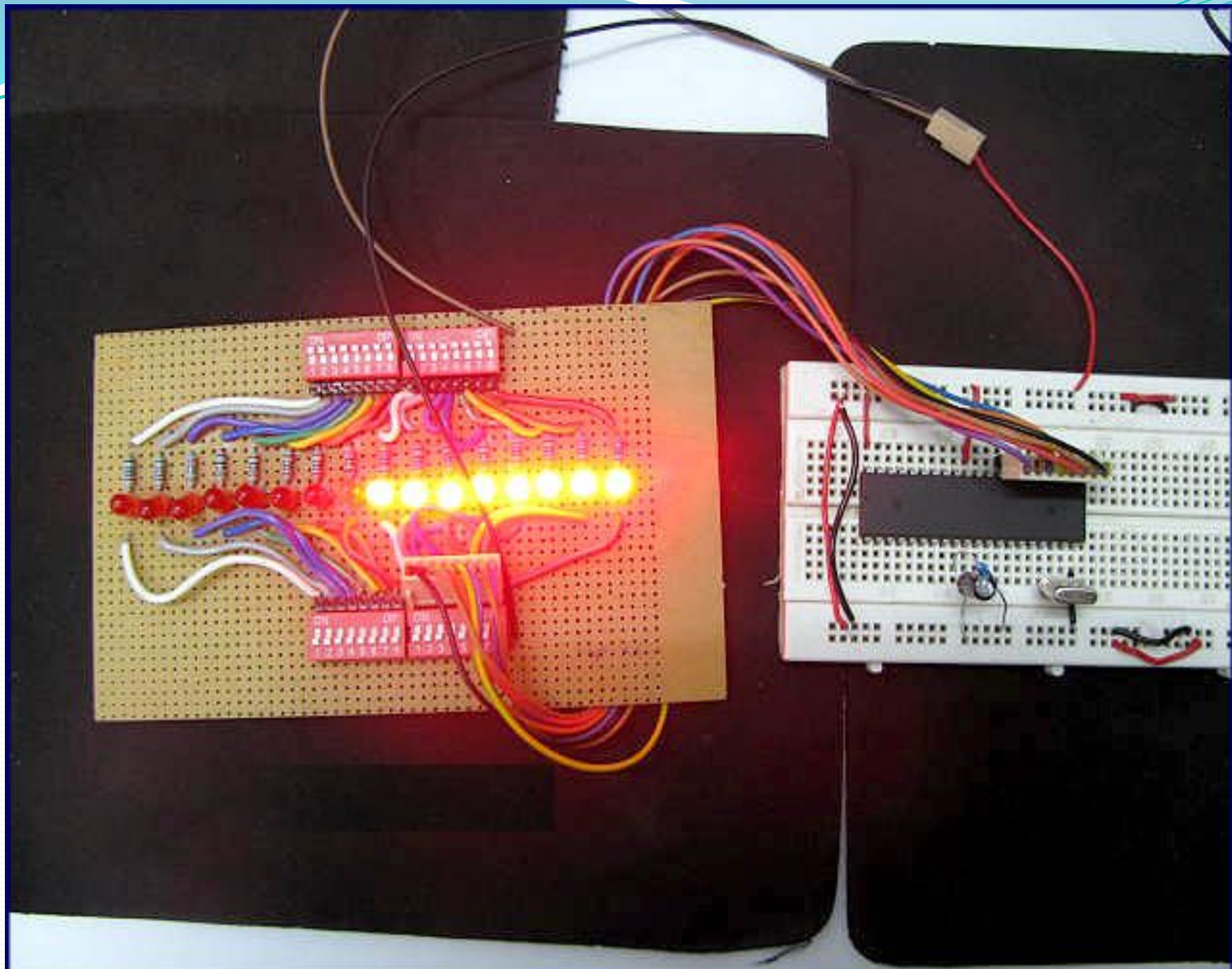
Program

```
// Program to blink the LEDs. LEDs are connected to port 0.

#include<reg51.h>

void delay(int time)    //This function produces a delay in msec.
{
    int i,j;
    for(i=0;i<time;i++)
        for(j=0;j<1275;j++);
}

void main()
{
    while(1)
    {
        P0=0x00;
        delay(50);
        P0=0xff;
        delay(50);
    }
}
```





Introduction to Keil and Proteus

Assembling and running an 8051 program

- [Document 55.pdf](#)

Embedded Operating system.

- An **embedded operating system (OS)** is a specialized **operating system** designed to perform a specific task for a device that is not a computer. An **embedded operating system's** main job is to run the code that allows the device to do its job.
- Embedded systems may use a ROM-based operating system or they may use a disk-based system, like a PC. But an embedded system is not usable as a commercially viable substitute for general purpose computers or devices.

What makes a good RTOS?

- Multi-threaded and preemptible
- Thread priority has to exist because no deadline driven OS exists
- Must support predictable thread synchronization mechanisms
- A system of priority inheritance must exist

Who are the Embedded OS players

- **Microsoft**

- Embedded NT/XP
 - “Real-time” control
- Windows CE (CE.NET)
 - Internet devices
- Pocket PC 2002
 - Handheld PC’s and PDA’s

- **Wind River Systems**

- VxWorks
- pSOS

Conti.....

- **Commercial Embedded Linux**

- AMIRIX Embedded Linux
- Coollogic Coollinux
- Coventive Xlinux
- Esfia RedBlue Linux
- KYZO Pizza Box Linux
- Lineo Embedix
- LynuxWorks BlueCat
- MontaVista Linux

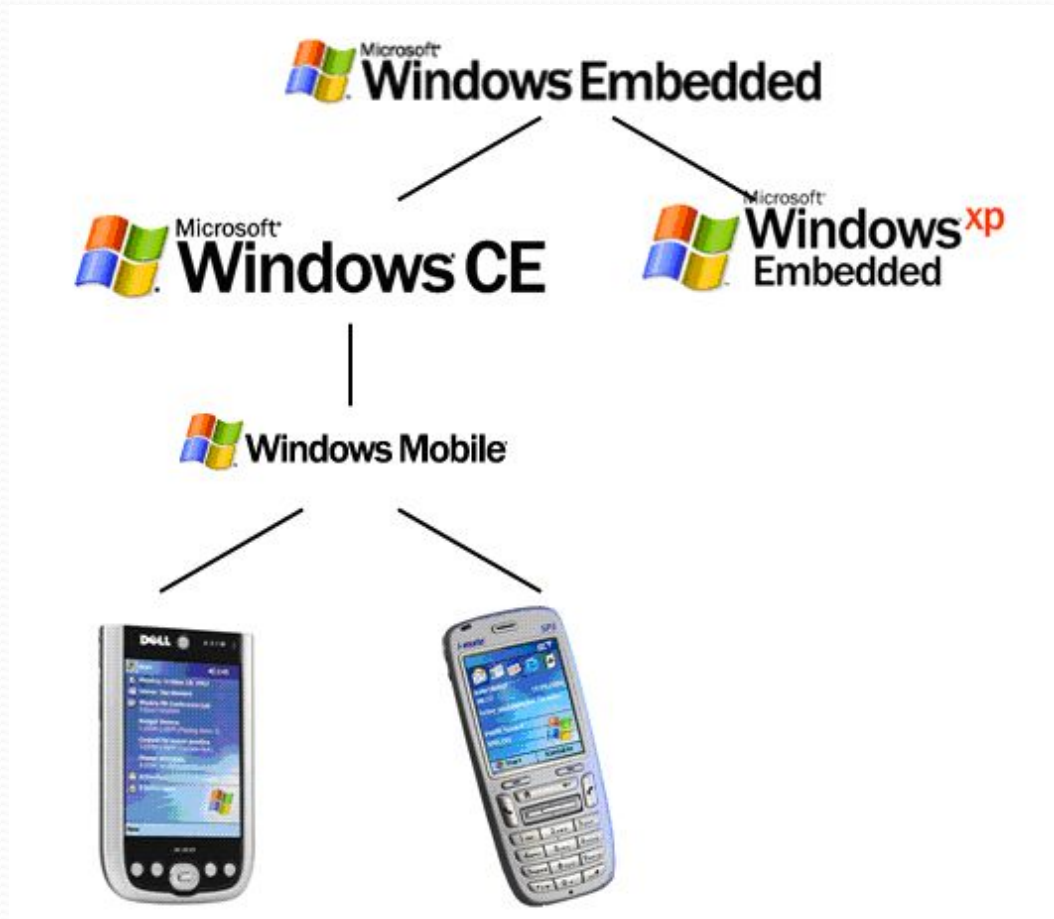
- **Open Source Embedded Linux**

- Embedded Debian Project
- ETLinux
- uCLinux
- uLinux (muLinux)

Conti...

- **QNX Software Systems**
 - QNX
- **Palm PDA**
 - Palm OS
- **Symbian**
 - Symbian OS
- **Green Hills Software**
 - Integrity

Windows Embedded Family



Some of the popular RToS are

- 1. QNX RTOS v 6.1
- 2. Windows CE 3.0
- 3. pSo System /x86 2.2.6
- 4. Vx works (wind River System)
- 5. Windows NT

Conti...

QNX RTOS v6.1

- ① Client-Server based Architecture.
- ② 10 K bytes of micro-Kernel surrounded by a team of optional processes higher-level OS services.
- ③ System performance is fast and predictable and is robust.
- ④ QNX has used in tiny ROM

Windows NT

- ① Overall architecture is good
- ② it is suitable RTOS for control system that need a second user interface

Windows CE 3.0

- ① rich in features
- ② it is available for a variety of hardware platforms
- ③ it shows True Real time behaviour most of the times.
- ④ System is complex and highly configurable
- ⑤ The system is robust and well handling of memory

PSO System / X86 2.2.6

- ① Psost is a small Kernel
- ② use Software bus to Communication between different modules
- ③ Psos has a Multiprocessor version of psost, which can
- ④ it use Master and Slave principle
- ⑤ it is used in available only for selected processors

- ③ it always Need Hardisk and powerful processors
- ④ for user interaction it Requires screen and Keyboard.

Vx works

- ① it is used in wide variety of target processors.
- ② it has set of powerful cross development tools
- ③ it has good Range of Communication software Options such as Ethernet or serial line for the target connection
- ④ it used wind micro Kernel which supports multitasking scheduling, intertasking management, memory management

Operating System Architecture

- 1. Monolithic operating system
- 2. Layered operating System
- 3. Client –Server operating System
- [RTOS architecture.pdf](#)

Design parameters of an embedded system and its significance

- All Embedded system contain a processor and software . The processor may be 8051 microcontroller or a pentium-IV processor(having a clock speed of 2.4 Ghz)
- All embedded system contain memory and input, output . For example in a microwave oven the inputs are the buttons on the front panel and a temperature probe and the output are human readable display and microwave radiation. Input to the system generally take the form of sensors and probes, communication signals or controls.

Conti...

- The common critical features and design requirements of an embedded hardware are

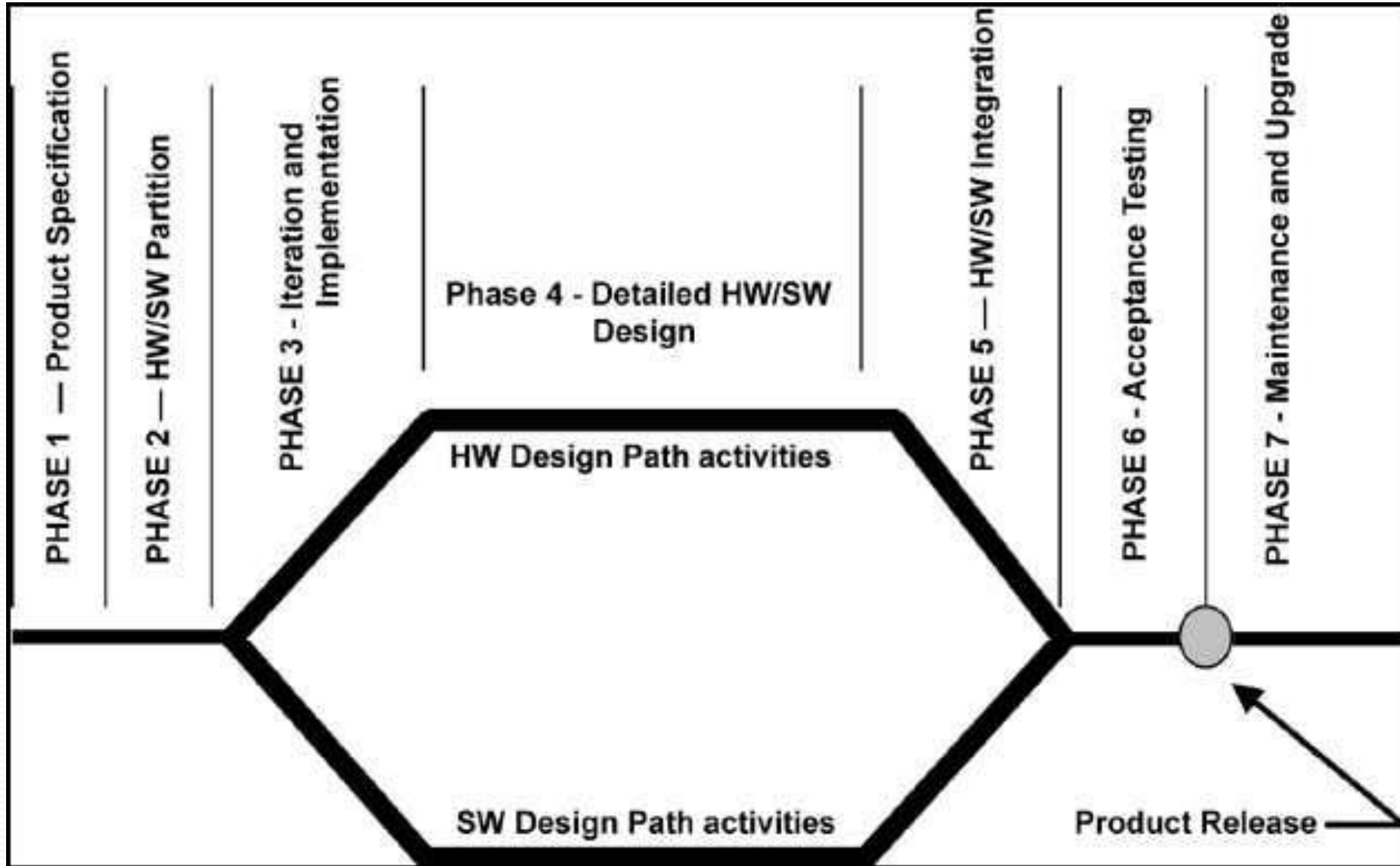
1. Processing Time
2. Throughput
3. Response
4. Memory
5. Power consumption
6. Number of units
7. Expected lifetime
8. Program Installation
9. Testing & Debugability
10. Reliability

EMBEDDED DESIGN LIFE CYCLE

- Unlike the design of a software application on a standard platform, the design of an embedded system implies that both software and hardware are being designed in parallel. Although this isn't always the case, it is a reality for many designs today. The profound implications of this simultaneous design process heavily influence how systems are designed.

- Embedded Product Development Life Cycle (Let us call it as EDLC, though it is not a standard and universal term) is an 'Analysis -Design -Implementation' based standard problem solving approach for Embedded Product Development. In any product development application, the first and foremost step is to figure out what product needs to be developed (analysis), next you need to figure out a good approach for building it (design) and last but not least you need to develop it (implementation). It provides a schematic representation of the embedded design life cycle (which has been shown ad nauseam in marketing presentations).

Conti...



Hardware and Software partitioning and co-design

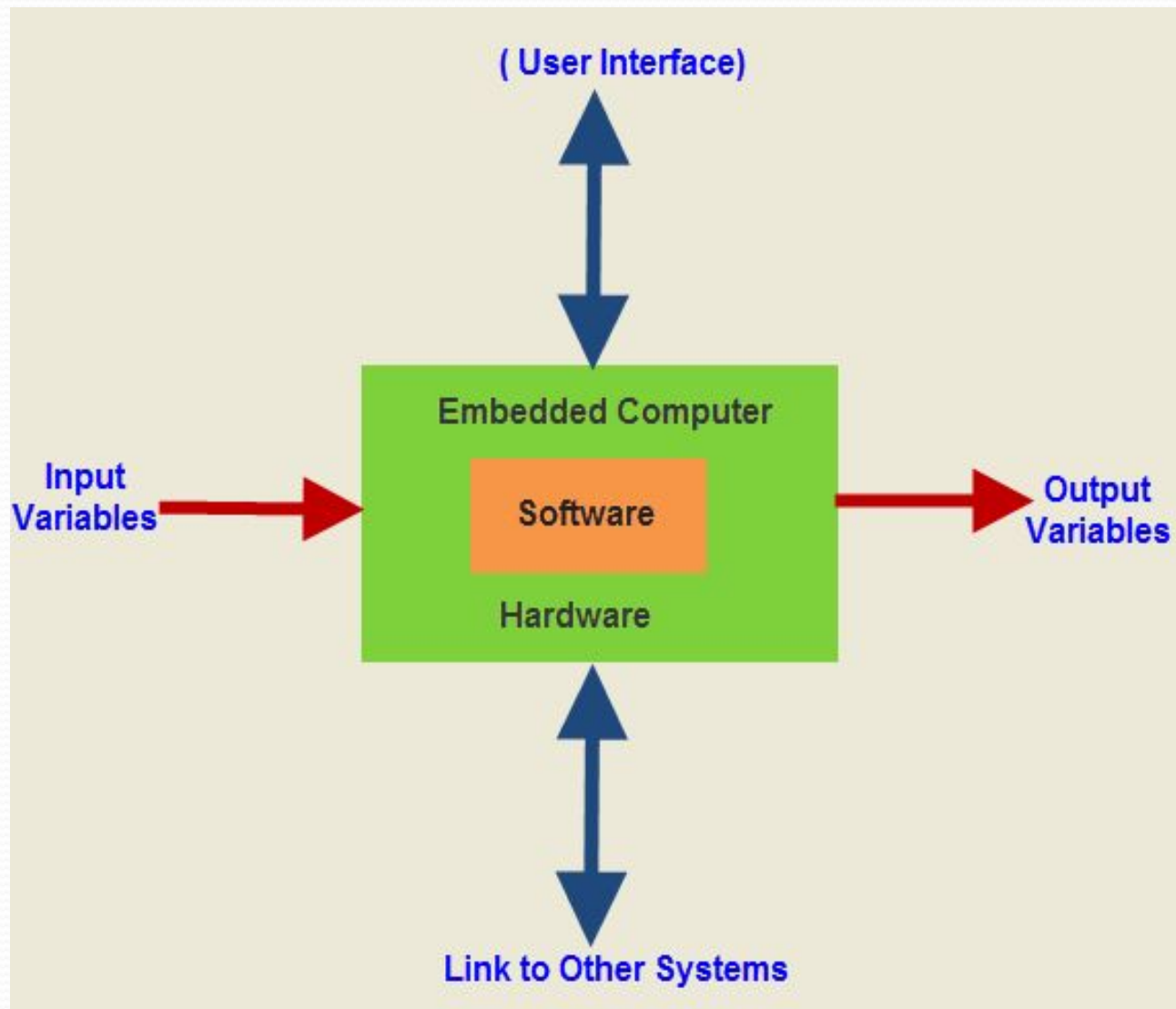
- The process of deciding, for each subsystem, whether the required functionality is more advantageously implemented in hardware or software.
- Partitioning into hardware and software affects overall system cost and performance
- Hardware implementation
 - Provides higher performance via hardware speeds and parallel execution of operations
 - Incurs additional expense of fabricating ASICs
- Software implementation
 - May run on high-performance processors at low cost (due to high-volume production)
 - Incurs high cost of developing and maintaining (complex) software

Partitioning Approaches

- Start with all functionality in software and move portions into hardware which are time-critical and can not be allocated to software (*software-oriented partitioning*)
- Start with all functionality in hardware and move portions into software implementation (*hardware-oriented partitioning*)
- [unit1.pdf](#)

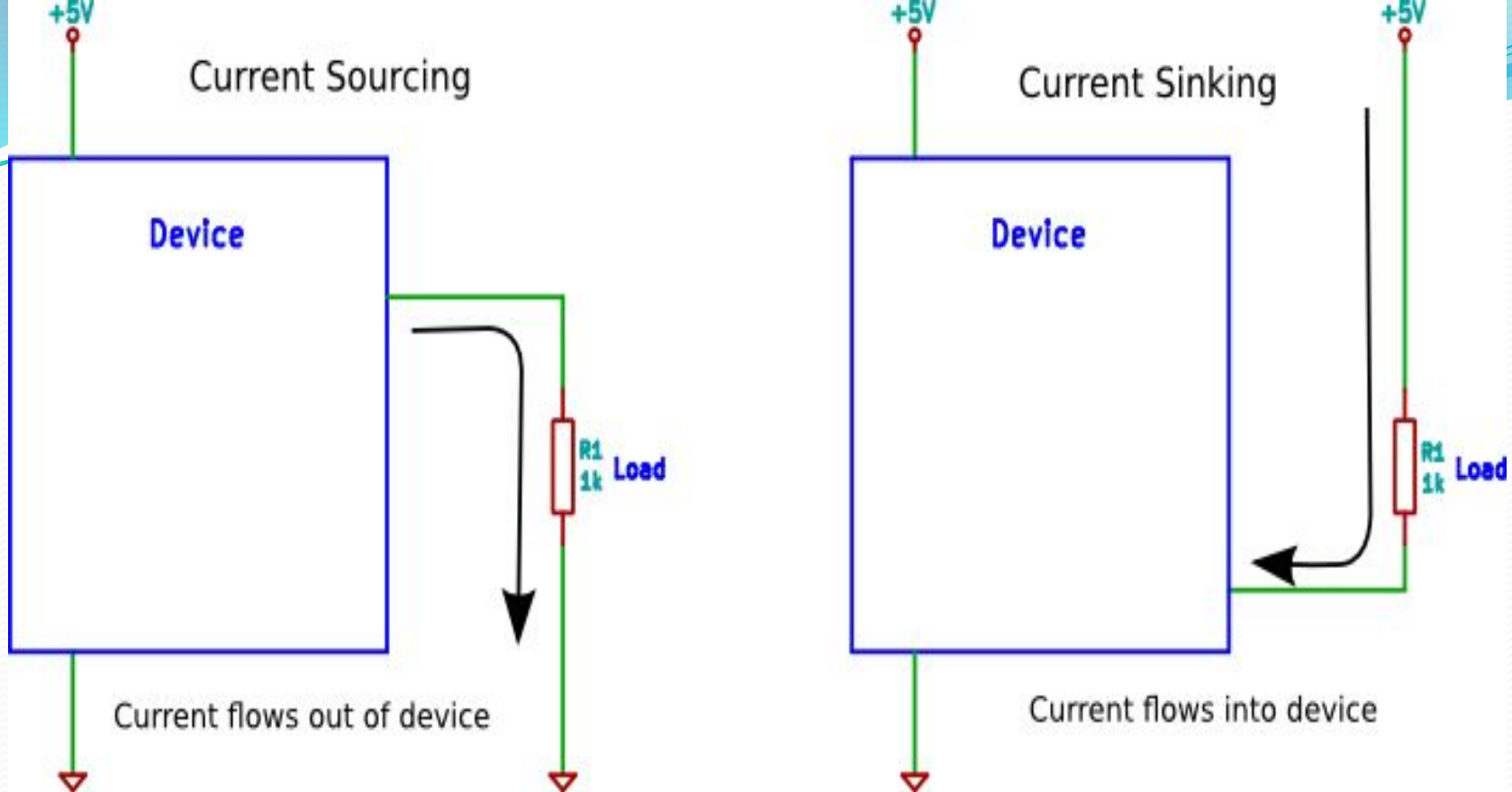
Hardware Fundamentals for the embedded developers.

- **Embedded system hardware** is built with a microprocessor or microcontroller. The **embedded system hardware** has elements like input output (I/O) interfaces, user interface, memory and the display. Usually, an **embedded system** consists of: Power Supply.



Sinking and Sourcing

- Current sourcing and sinking refers to the way that an external load is connected to a circuit, system, microcontroller or other electronic device.
- **Current Sourcing**
- When a load is connected to a device so that the device supplies current to the load (sources current) then the configuration is said to be current sourcing.
- An example is a series resistor and LED connected between a microcontroller pin and GND. When the microcontroller pin is switched high (logic 1) then the microcontroller will source current to the load. In this configuration a logic 1 will supply power to the load (switch the LED on) and a logic 0 will switch power to the load off (switch the LED off).

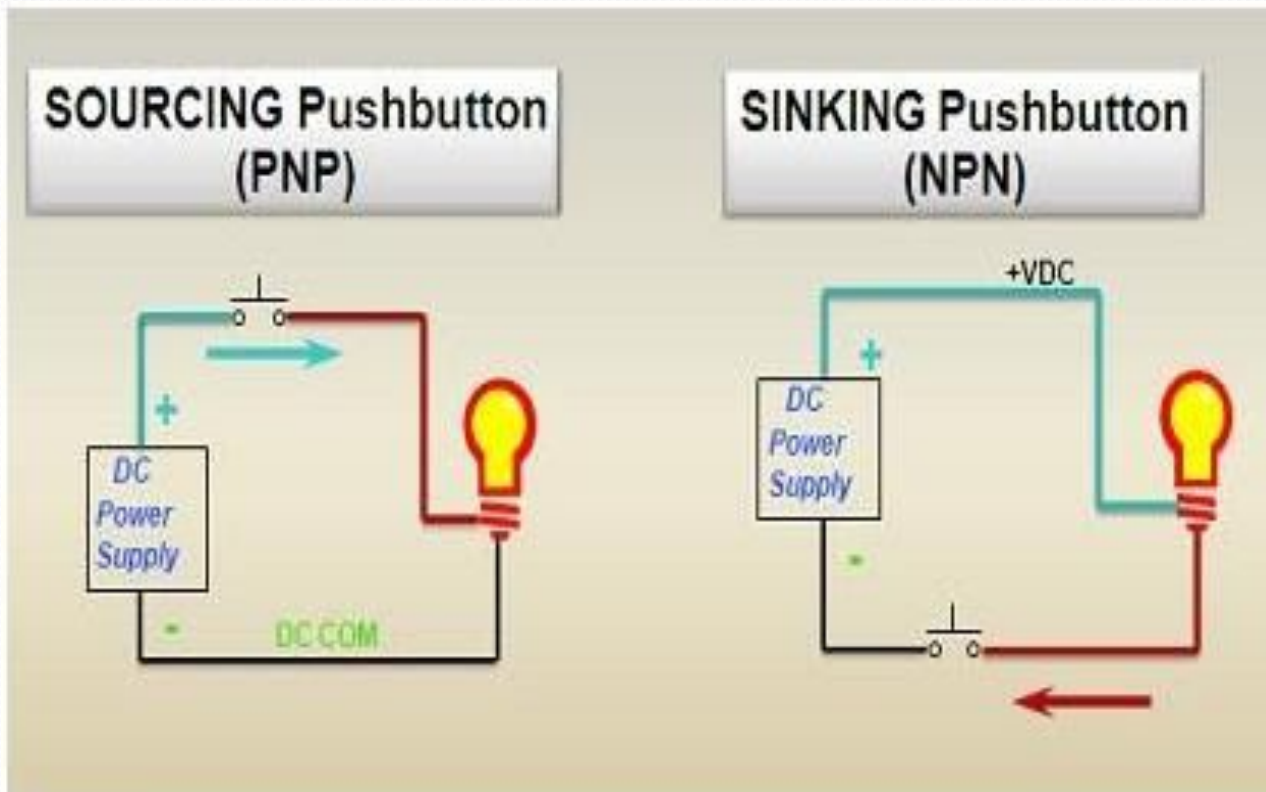


- **Current Sinking**

- When a load is connected to a device so that current flows from the power supply through the load and into the device, then the configuration is said to be current sinking. When current flows into the device, it is said to be sinking current.

Conti....

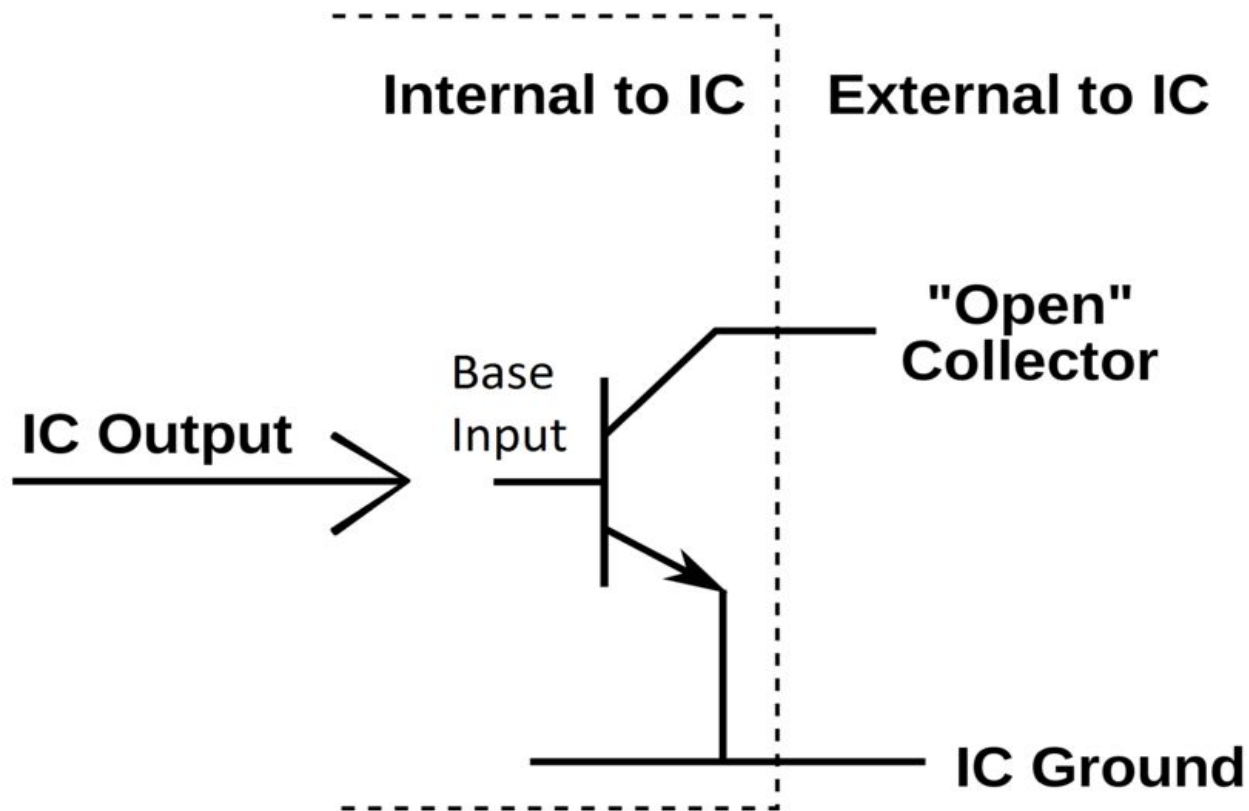
An example of current sinking is when a series resistor and LED is connected between power (e.g. +5V) and a microcontroller pin. When the microcontroller pin is switched high (logic 1) then the current to the load is switched off. When the microcontroller switches the pin low (logic 0), current flows through the load.



Open collector outputs

- An open collector is a common type of output found on many integrated circuits (IC), which behaves like a switch that is either connected to ground or disconnected. Instead of outputting a signal of a specific voltage or current, the output signal is applied to the base of an internal NPN transistor whose collector is externalized (open) on a pin of the IC. The emitter of the transistor is connected internally to the ground pin. If the output device is a MOSFET the output is called open drain and it functions in a similar way. For example, the I²C bus is based on this concept.

Conti...

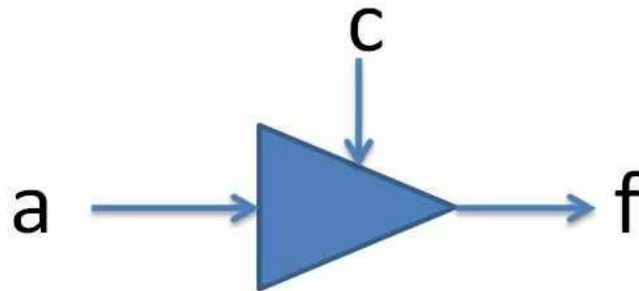


Tristate output

- **tri-state output (three-state output)** An electronic **output** stage consisting of a logic gate, commonly an inverter or buffer, that exhibits three possible logic states, namely logic 1, logic 0, and an inactive (high-impedance or open-circuit) state.

Conti...

Tri-State Buffer



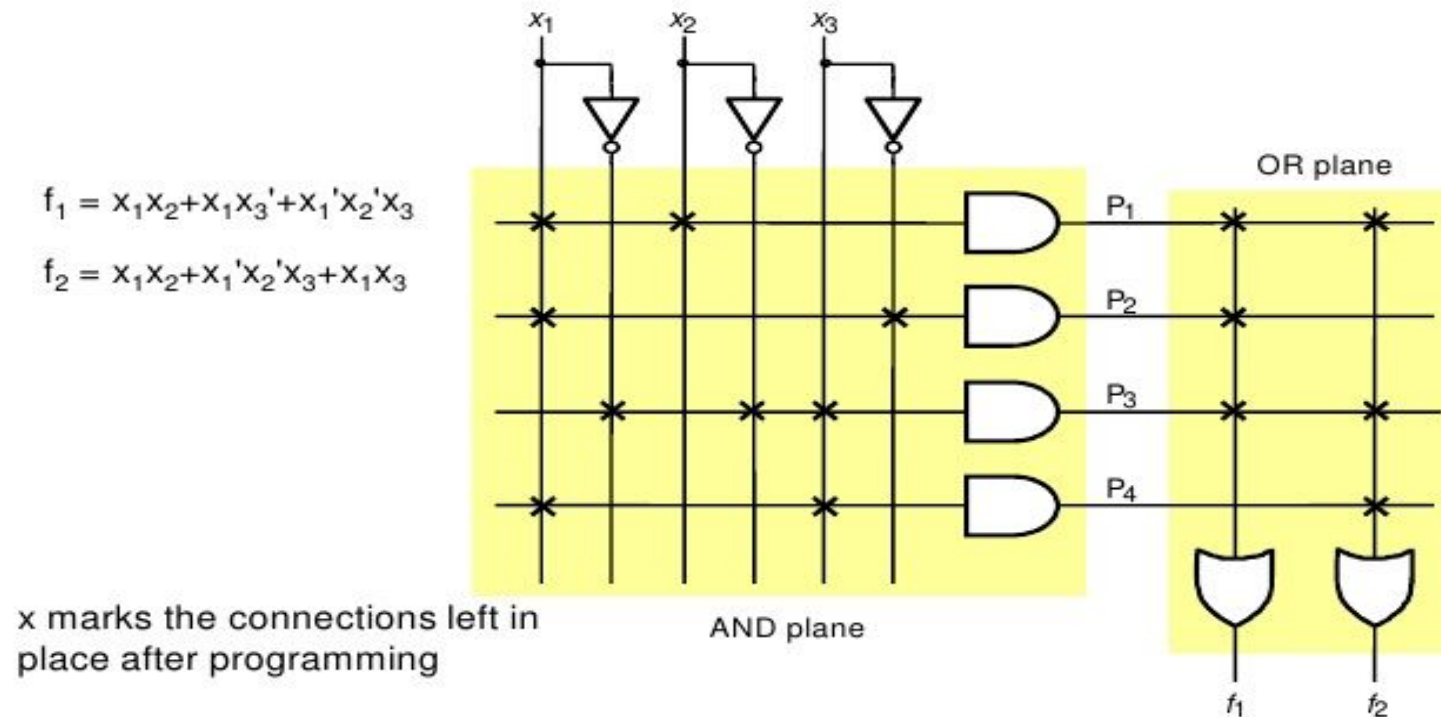
c	a	f
0	0	<u>Z</u>
0	1	Z
1	0	0
1	1	1

PLD's

- Short for programmable logic device, a generic term for an integrated circuit that can be programmed in a laboratory to perform complex functions. A **PLD** consists of arrays of AND and OR gates. A system designer implements a logic design with a device programmer that blows fuses on the **PLD** to control gate operation.
- **Main types of PLD's are**
 1. PLA
 2. PAL
 3. ROM
 4. CPLD
 5. FPGA

Conti...

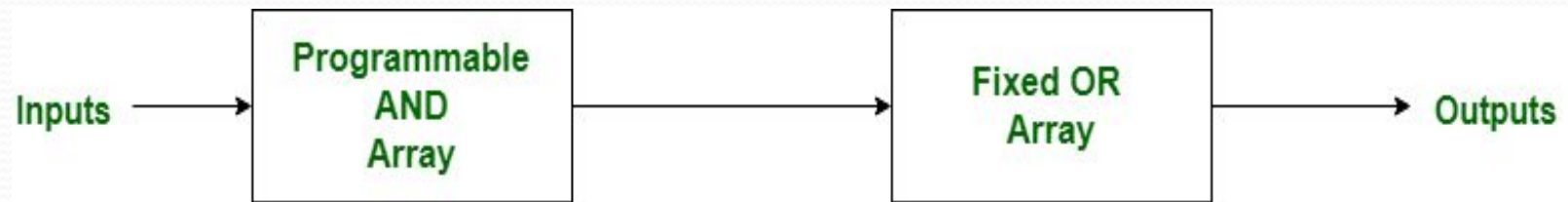
► Customary Schematic of a PLA



Difference between Programmable Logic Array and Programming Array Logic

- Programmable Logic Array (PLA) and Programming Array Logic (PAL) are the categories of programming logic device.
- In PLA or Programmable Logic Array, there are massive functions can be implemented. Whereas in PAL or Programmable Array Logic, there is finite functions can be implemented.
The distinction between PLA and PAL is that, PLA have programmable AND array and fixed OR array. On the other hand, PAL have programmable AND array and programming OR array.

Conti...



PLA

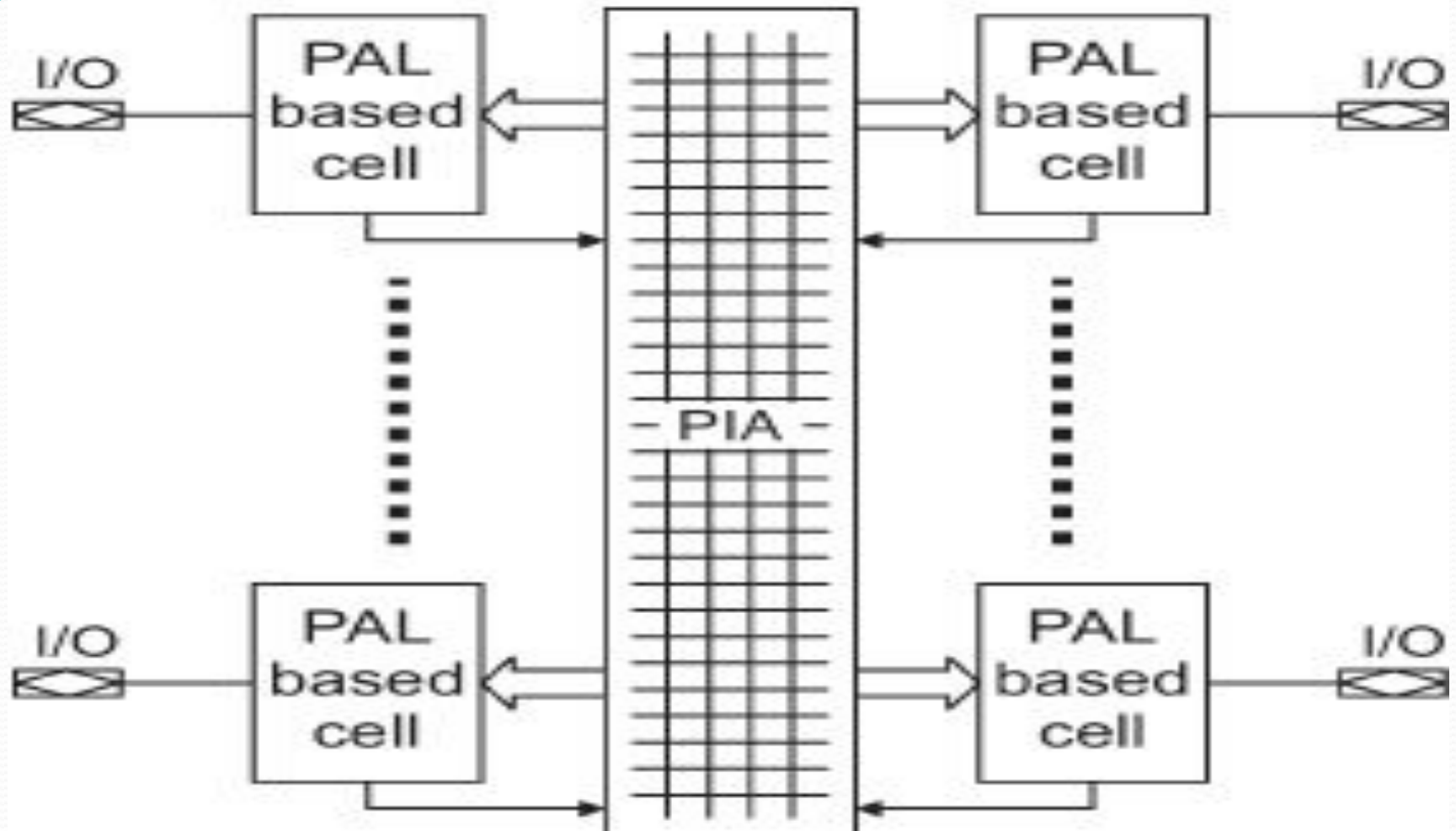


PAL

CPLD

- A complex programmable logic device (**CPLD**) is a programmable logic device with complexity between that of PALs and FPGAs, and architectural features of both. The main building block of the **CPLD** is a macro cell, which contains logic implementing disjunctive normal form expressions and more specialized logic operations.

Conti....



● CPLD Architecture

FPGA

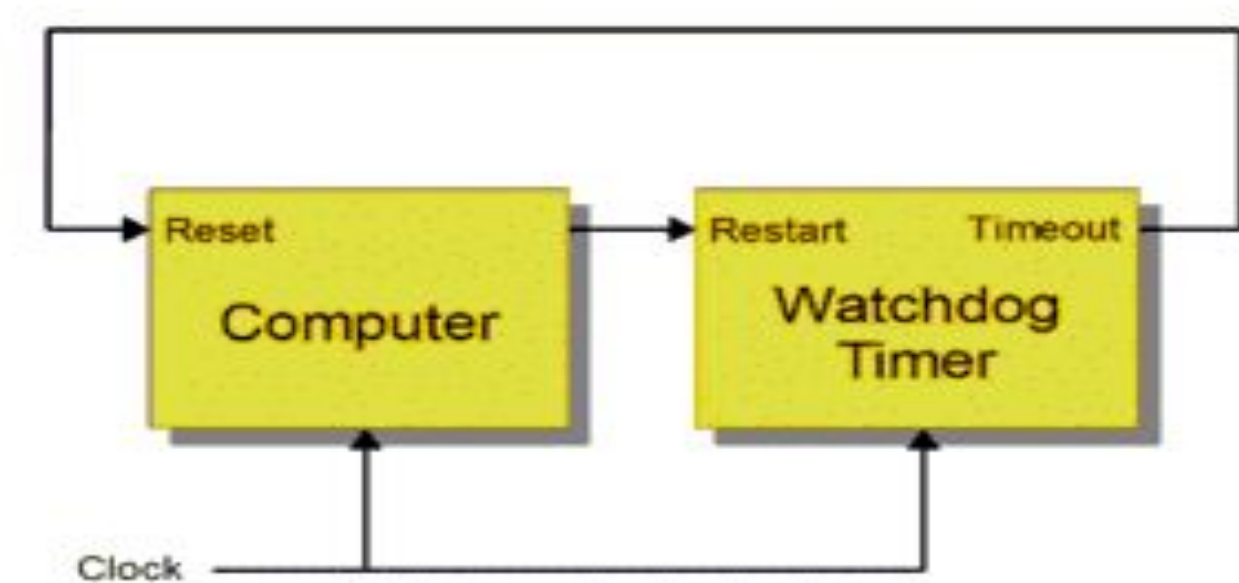
- Field Programmable Gate Arrays (**FPGAs**) are semiconductor devices that are based around a matrix of configurable logic blocks (CLBs) connected via programmable interconnects. **FPGAs** can be reprogrammed to desired application or functionality requirements after manufacturing.



Watch dog Timer

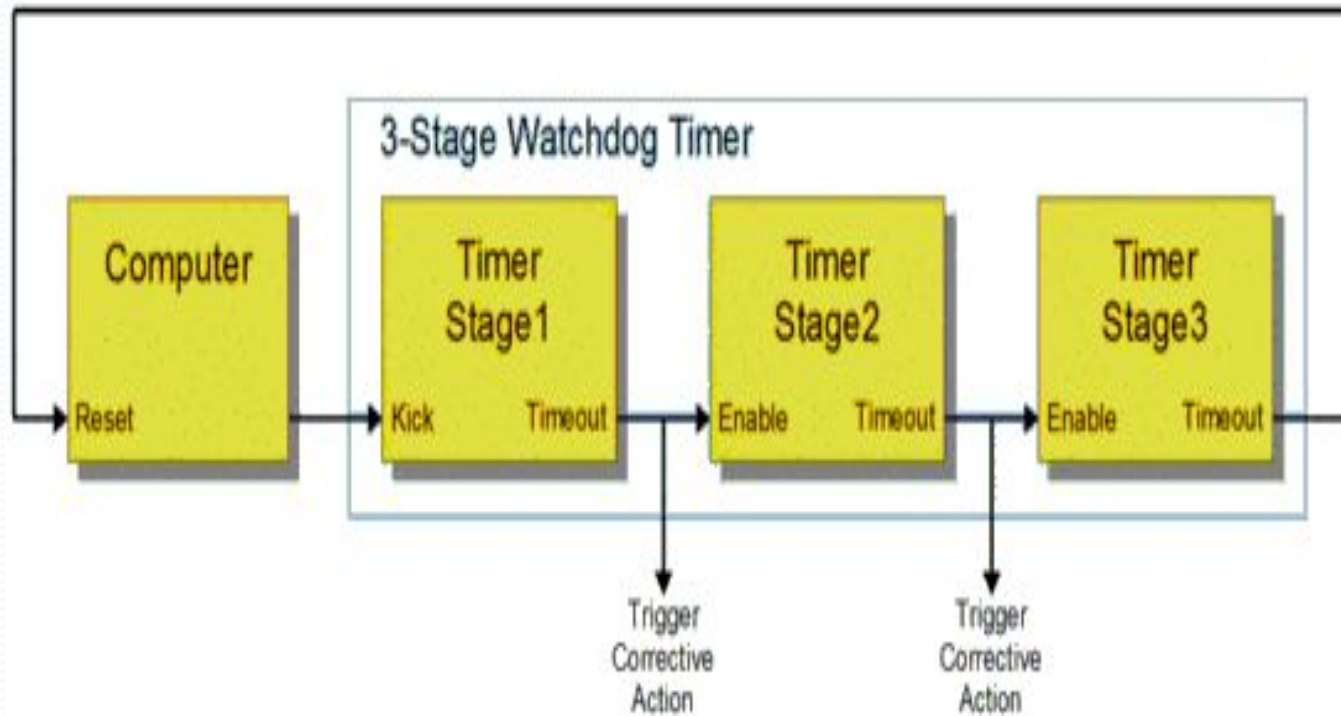
- A **watchdog timer** (sometimes called a computer operating properly or COP **timer**, or simply a **watchdog**) is an electronic or software **timer** that is **used** to detect and recover from computer malfunctions. During normal operation, the computer regularly resets the **watchdog timer** to prevent it from elapsing, or "timing out".
- Watchdog timers are commonly found in embedded systems and other computer-controlled equipment where humans cannot easily access the equipment or would be unable to react to faults in a timely manner. In such systems, the computer cannot depend on a human to invoke a reboot if it hangs.

Conti...



Single stage watch dog Timer

Conti...



● Multistage Watchdog Timer